Contents

GhostWire Bible **10**
GhostWire Logo . . . 10
Table of Contents . . . 10
Index (Selected Topics) . . . 10
How to Use This Bible . . . 11

GhostWire Project Overview **11**
Executive Summary . . . 11
Table of Contents . . . 11
1. What is GhostWire? . . . 11
2. Why Mesh Networking? . . . 12
3. Real-World Scenarios . . . 12
Communities with Limited Connectivity . . . 12
Disaster Response . . . 12
Rural Communities . . . 12
Community Events . . . 12
4. Key Features & Benefits . . . 12
5. How GhostWire Works (Plain & Technical) . . . 12
Plain-Language Overview . . . 12
The Device Hierarchy . . . 13
AI Routing . . . 13
Technical Deep Dive . . . 13
6. Visual Guide: GhostWire in Action . . . 13
7. Best Practices & Anti-Patterns . . . 13
Best Practices . . . 13
Anti-Patterns . . . 14
8. Project Status (September 2026) . . . 14
Transports Working Today . . . 14
Protocol Bridges . . . 14
External Security Audit . . . 14
New Chapters . . . 14
9. Frequently Asked Questions . . . 14
10. Further Reading & Resources . . . 15
Appendix: Glossary . . . 15
End of Chapter . . . 15

Getting Started with GhostWire **15**
Table of Contents . . . 15
1. Welcome & Audience . . . 16
2. What You Can Do with GhostWire . . . 16
3. Quick Start (No Coding) . . . 16
Option 1: Web Demo . . . 16
Option 2: Mobile/Desktop App . . . 16
4. Full Setup (Technical) . . . 16
Prerequisites . . . 16
Step-by-Step . . . 16

5. Choosing Your Role and Transport . . . 17
Battery States and Privacy . . . 17
Pairing with QR Codes . . . 17
6. Joining & Creating a Mesh . . . 17
7. Using the Web UI & CLI . . . 17
Web UI . . . 17
CLI (for Power Users) . . . 18
8. Troubleshooting & Support . . . 18
9. Real-World Onboarding Scenarios . . . 18
Community Event . . . 18
Disaster Response . . . 18
Rural Village . . . 18
10. Best Practices for New Users . . . 19
11. Further Reading & Resources . . . 19
End of Chapter . . . 19

Architecture Deep Dive **19**
Table of Contents . . . 19
1. Overview & Philosophy . . . 20
2. System Diagram & Visuals . . . 20
3. Core Components (Plain & Technical) . . . 20
Plain-Language . . . 20
Technical . . . 20
4. Device Hierarchy and Supernodes . . . 21
Device Classes . . . 21
Roles . . . 21
Supernode Election . . . 21
Trust and Relay Authority . . . 21
Hardware Ladder and ESP32 Gateway Bridge . . . 21
Mailbox Pattern for Leaf Devices . . . 22
5. Transports Layer . . . 22
6. Protocol Adapters Layer . . . 22
7. AI Routing Layer . . . 22
Layer 1: LightGBM Per-Message Classification . . . 22
Layer 2: Gemma 4 Analytical Pass . . . 23
Layer 3: GNN Graph Router . . . 23
Layer 4: PRoPHET Delay-Tolerant Routing . . . 23
Model Ladder . . . 23
Telemetry and Model Distribution . . . 23
Store-and-Forward DTN Integration . . . 23
8. Security & Trust Layer . . . 23
9. Store & Forward / Federation . . . 24
10. Data Flow: Message Lifecycle . . . 24
11. Deployment Blueprints . . . 25
Home/Personal Mesh . . . 25
Community/Neighborhood Mesh . . . 25
Disaster/Field Deployment . . . 25
Enterprise/Federated Mesh . . . 25

12. Best Practices & Anti-Patterns . . . 25
Best Practices . . . 25
Anti-Patterns . . . 25
13. Glossary & Reference . . . 26
End of Chapter . . . 26

Transports & Protocols **26**
Table of Contents . . . 26
1. What is a Transport? . . . 26
2. Supported & Planned Transports . . . 27
3. Traffic Armor and Auto-Switching . . . 27
4. Transport Selection: Thompson Sampling . . . 27
5. How Transports Work (Plain & Technical) . . . 28
Plain-Language . . . 28
Technical . . . 28
6. Which Transport When . . . 28
7. Real-World Use Cases . . . 29
Disaster Response . . . 29
Urban Mesh . . . 29
Rural Connectivity . . . 29
Connectivity in Constrained Environments . . . 29
8. Configuration & Code Examples . . . 29
Enabling/Disabling Transports (Config) . . . 29
Using Transports in Code . . . 30
Web UI Example . . . 30
9. Visual Guide: Transport Topologies . . . 30
10. Best Practices & Anti-Patterns . . . 30
Best Practices . . . 30
Anti-Patterns . . . 30
11. Troubleshooting & FAQ . . . 31
12. Further Reading & Resources . . . 31
End of Chapter . . . 31

Security & Privacy **31**
Table of Contents . . . 31
1. Executive Summary . . . 32
2. Security Foundations (Plain & Technical) . . . 32
3. Threat Models & Real-World Risks . . . 32
4. Security Architecture & Modules . . . 32
5. Encryption & Key Management . . . 32
6. Modern Crypto & Identity Stack . . . 33
Post-Quantum Key Encapsulation . . . 33
Symmetric Encryption . . . 33
Key Exchange and Forward Secrecy . . . 33
Duress PIN and Coercion Resistance . . . 33
Identity: DIDs, Shamir Secret Sharing, Dead Man's Switch . . . 33
Privacy Tiers (L0-L3) . . . 34
SPHINX Unlinkable Replies and Cover Traffic . . . 34

7. Sybil Defense & Trust . . . 34
8. Quotas, Blacklists, and Abuse Prevention . . . 35
9. Traffic Obfuscation & Anti-Analysis . . . 35
10. Disaster & Infrastructure Failure Scenarios . . . 35
11. Best Practices & Anti-Patterns . . . 35
Best Practices . . . 35
12. Actionable Security Checklists . . . 35
13. Visuals: Security Layers & Flows . . . 36
14. FAQ & Troubleshooting . . . 36
15. External Audit Status . . . 36
16. Further Reading & Resources . . . 37
End of Chapter . . . 37

Protocol Adapters **37**
Table of Contents . . . 37
1. What is a Protocol Adapter? . . . 37
2. Supported & Planned Adapters . . . 38
3. How Adapters Work (Plain & Technical) . . . 38
Plain-Language . . . 38
Technical . . . 38
4. The Adapter Registry . . . 38
5. Real-World Bridging Scenarios . . . 39
Disaster Response . . . 39
Activist Network . . . 39
Rural Community . . . 39
6. Configuration & Code Examples . . . 39
Enabling/Disabling Adapters (Config) . . . 39
Using Adapters in Code . . . 39
Web UI Example . . . 39
7. Visual Guide: Adapter Topologies . . . 39
8. Companion Projects . . . 40
9. Developer Notes & API Reference . . . 40
10. Best Practices & Anti-Patterns . . . 40
Best Practices . . . 40
Anti-Patterns . . . 40
11. Troubleshooting & FAQ . . . 40
12. Further Reading & Resources . . . 41
End of Chapter . . . 41

Frequently Asked Questions (FAQ) **41**
Table of Contents . . . 41
1. General Questions . . . 41
2. Getting Started . . . 42
3. Technical & Developer Questions . . . 42
4. Security & Privacy . . . 43
5. Real-World Scenarios . . . 43
6. Troubleshooting & Support . . . 44
7. Visuals: Common Flows . . . 44

8. Further Reading & Resources . . . 44
End of Chapter . . . 44

Contributing & Community **44**
Table of Contents . . . 44
1. Welcome & Philosophy . . . 45
2. How Anyone Can Contribute . . . 45
3. Code Contributions (Technical) . . . 45
4. Writing, Design, and Outreach . . . 45
5. Community Guidelines & Code of Conduct . . . 45
6. Example Contribution Journey . . . 46
7. Visuals: Contribution Flow . . . 46
8. Best Practices for Contributors . . . 46
9. Further Reading & Resources . . . 46
End of Chapter . . . 47

Deployment & Operations **47**
Table of Contents . . . 47
1. Overview . . . 47
2. Deployment Scenarios . . . 47
Home/Personal Mesh . . . 47
Community/Neighborhood Mesh . . . 47
Disaster/Field Deployment . . . 48
Enterprise/Federated Mesh . . . 48
3. Assigning Roles by Power and Hardware . . . 48
Device Classes . . . 48
Roles . . . 48
Role Assignment Principles . . . 48
Battery States and Role Behavior . . . 49
Solar and Battery-Powered LoRa Relays . . . 49
Supernode Election Operations . . . 49
ESP32 Gateway-Bridged Deployments . . . 49
OEM Battery Optimization Note . . . 49
4. Step-by-Step Deployment Guides . . . 50
Local (Laptop/Desktop) . . . 50
Docker . . . 50
Cloud (AWS, GCP, Azure) . . . 50
Bare-Metal . . . 50
5. Docker, Cloud, and Bare-Metal Installations . . . 50
6. Monitoring, Logging, and Maintenance . . . 50
7. Visuals: Deployment Topologies . . . 51
8. Troubleshooting & Recovery . . . 51
9. Best Practices & Anti-Patterns . . . 51
Best Practices . . . 51
10. Further Reading & Resources . . . 52
End of Chapter . . . 52

Advanced Security & Threat Response **52**

Table of Contents . . . 52
1. Overview . . . 52
2. Threat Modeling (Plain & Technical) . . . 52
3. Incident Response & Forensics . . . 53
Duress PIN Incident Response . . . 53
4. Security Module Configuration . . . 53
5. Audit & Compliance . . . 54
External Audit Status . . . 54
6. Example Attack/Defense Scenarios . . . 54
7. Visuals: Threat Flows & Response . . . 55
8. Actionable Checklists . . . 55
9. Further Reading & Resources . . . 55
End of Chapter . . . 55

Performance & Scaling **55**
Table of Contents . . . 56
1. Overview . . . 56
2. Optimization Tips . . . 56
3. Benchmarks & Metrics . . . 56
4. Tiered Scaling Architecture . . . 57
The Problem with Flat Meshes . . . 57
Supernode and Relay Hub Election . . . 57
Regional DHT Sharding (v3/v4 Plan) . . . 57
Scaling Targets . . . 57
5. Energy-Aware Transport Selection . . . 57
Energy-Efficiency Weights . . . 57
How Thompson Sampling Works . . . 58
Battery-Aware Behavior . . . 58
6. AI Latency Context . . . 58
Inference Timings . . . 58
AI Architecture . . . 58
Telemetry . . . 59
7. Troubleshooting Performance . . . 59
8. Visuals: Performance Flows . . . 59
9. Best Practices & Anti-Patterns . . . 59
Best Practices . . . 59
10. Further Reading & Resources . . . 60
End of Chapter . . . 60

Developer Guide **60**
Table of Contents . . . 60
1. Overview . . . 60
2. Extending GhostWire (Transports, Adapters, Modules) . . . 60
Adding a New Transport . . . 60
Adding a Protocol Adapter . . . 61
Adding a Security Module . . . 61
3. Adding a Bridge Adapter . . . 61
Reference: Matrix Adapter . . . 61

Reference: Meshtastic Adapter . . . 61
4. Extending AI Routing . . . 61
5. API Reference & Hooks . . . 62
6. Plugin System & Architecture . . . 62
7. Testing & CI/CD . . . 62
8. Visuals: Developer Flows . . . 63
9. Best Practices & Anti-Patterns . . . 63
Best Practices . . . 63
Anti-Patterns . . . 63
10. Further Reading & Resources . . . 63
End of Chapter . . . 63

Case Studies & Scenario Narratives **63**
Table of Contents . . . 63
1. Overview . . . 64
2. Hurricane Response Within 200km . . . 64
3. Community Blackout . . . 65
4. Rural Village Connectivity . . . 65
5. Festival Mesh . . . 66
6. Lessons Learned . . . 67
7. Visuals: Scenario Topologies . . . 67
8. Best Practices & Takeaways . . . 67
9. Further Reading & Resources . . . 68
End of Chapter . . . 68

Glossary & Reference **68**
Table of Contents . . . 68
1. Glossary . . . 68
2. Acronyms & Definitions . . . 71
3. Quick Reference: Commands & Config . . . 71
CLI Commands . . . 71
Config Example . . . 72
4. Troubleshooting Table . . . 72
5. Visuals: Reference Flows . . . 72
6. Cross-References . . . 72
7. Further Reading & Resources . . . 72
End of Chapter . . . 73

15. Device Hierarchy and Scaling **73**
Table of Contents . . . 73
1. Executive Summary . . . 73
2. Why a Hierarchy Exists . . . 73
Plain-Language Explanation . . . 73
Technical Depth . . . 74
3. The Five Device Classes . . . 74
Plain-Language Explanation . . . 74
Technical Depth . . . 74
4. Node Roles Across Four Dimensions . . . 75

Plain-Language Explanation . . . 75
Technical Depth . . . 75
Critical Honesty About Relay Authority . . . 76
5. The Mailbox Pattern and Leaf Devices . . . 76
Plain-Language Explanation . . . 76
Technical Depth . . . 76
6. Why a Flat Mesh Fails . . . 76
Plain-Language Explanation . . . 76
Technical Depth . . . 77
7. Supernodes and Relay Hubs . . . 77
Plain-Language Explanation . . . 77
Technical Depth . . . 77
8. The Hardware Ladder . . . 78
Plain-Language Explanation . . . 78
Technical Depth . . . 78
9. Battery States and Device Behavior . . . 78
Plain-Language Explanation . . . 78
Technical Depth . . . 79
OEM Battery Considerations . . . 79
10. Privacy Tiers by Device . . . 79
Plain-Language Explanation . . . 79
Technical Depth . . . 79
11. Deployment Sizing Benchmarks . . . 80
Plain-Language Explanation . . . 80
Technical Depth . . . 80
12. A Practical Worked Example . . . 81
Plain-Language Explanation . . . 81
Technical Depth . . . 81
13. Best Practices and Anti-Patterns . . . 81
Plain-Language Explanation . . . 81
Anti-Patterns . . . 82
14. Glossary . . . 82
15. Further Reading . . . 82
End of Chapter . . . 83

16. AI Routing and Intelligence **83**
Table of Contents . . . 83
1. Executive Summary . . . 83
2. The Four-Layer AI Routing Stack . . . 84
3. Layer 1: LightGBM Per-Message Classifier . . . 84
Plain-Language . . . 84
Technical Depth . . . 84
4. Layer 2: Gemma 4 Edge Model . . . 85
Plain-Language . . . 85
Technical Depth . . . 86
5. Layer 3: Graph Neural Network (GNN) . . . 86
Plain-Language . . . 86
Technical Depth . . . 86

6. Layer 4: PRoPHET Delay-Tolerant Routing . . . 87
Plain-Language . . . 87
Technical Depth . . . 87
7. Model Ladder per Hardware . . . 87
8. Model Update Flow and Training Pipeline . . . 88
Plain-Language . . . 88
Technical Depth . . . 88
9. Thompson Sampling Transport Selection . . . 89
Plain-Language . . . 89
Technical Depth . . . 89
10. Privacy and Local AI Decisions . . . 89
Plain-Language . . . 89
11. Knowledge Ark and Anomaly Detection . . . 90
Plain-Language . . . 90
Technical Depth . . . 90
12. Worked Example: A Message's Journey . . . 90
Plain-Language . . . 90
Technical Depth . . . 90
13. Future Scopes . . . 91
Plain-Language . . . 91
Technical Depth . . . 91
14. End of Chapter . . . 92
End of Chapter . . . 92

17. Target Users **92**
Table of Contents . . . 92
Executive Summary . . . 92
1. The Six Target Personas . . . 93
1. Disaster Responders . . . 93
2. Privacy Advocates . . . 94
3. Rural Communities . . . 94
4. Activists and Journalists . . . 94
5. Whistleblowers . . . 95
6. Developers and Researchers . . . 95
2. Persona-to-Capability Map . . . 95
3. Four Illustrative Scenarios . . . 97
Scenario 1: Community Blackout with Protesters . . . 97
Scenario 2: Hurricane Disaster Response . . . 97
Scenario 3: Rural Village Connectivity . . . 97
Scenario 4: Festival . . . 97
4. Who Is Behind GhostWire . . . 97
5. Which Persona Should Read Which Chapter . . . 98
End of Chapter . . . 99

GhostWire Bible

The Comprehensive Guide to Secure, Modular Mesh Networking

Version 2.0, September 2026

GhostWire Logo

Table of Contents

1. Project Overview
2. Getting Started
3. Architecture Deep Dive
4. Transports & Protocols
5. Security & Privacy
6. Protocol Adapters
7. FAQ
8. Contributing & Community
9. Deployment & Operations
10. Advanced Security & Threat Response
11. Performance & Scaling
12. Developer Guide
13. Case Studies & Real-World Stories
14. Glossary & Reference
15. Device Hierarchy & Power-Aware Scaling
16. AI Routing & On-Device Intelligence
17. Who GhostWire Serves
18. Security Architecture (Technical Appendix)

Index (Selected Topics)

- **Security & Privacy:** 05, 10, 18 (see Security Architecture Appendix for deep technical details)
- **Deployment & Operations:** 09
- **Advanced Security & Threat Response:** 10
- **Developer Guide:** 12
- **Case Studies:** 13
- **Glossary & Reference:** 14
- **Technical Appendix:** 18 (Security Architecture)

How to Use This Bible

- **Non-technical readers:** Start with chapters 1-8 for plain-language explanations and real-world scenarios. Chapters 15-17 cover device hierarchy, AI routing, and the people GhostWire serves, all in accessible language.
- **Technical readers:** See chapters 3, 5, 10, 12, 15, and 16 for deep dives into architecture, security, performance, developer guidance, device tiers, and AI routing logic.

GhostWire: Communication for everyone, everywhere, every time.

GhostWire Project Overview

Executive Summary

GhostWire is a modular, privacy-focused mesh networking and messaging platform designed for everyone, from disaster responders and journalists to rural communities and tech enthusiasts. It enables secure, decentralized communication even when the internet is down or unavailable.

Table of Contents

1. What is GhostWire?
2. Why Mesh Networking?
3. Real-World Scenarios
4. Key Features & Benefits
5. How GhostWire Works (Plain & Technical)
6. Visual Guide: GhostWire in Action
7. Best Practices & Anti-Patterns
8. Frequently Asked Questions
9. Further Reading & Resources

---

1. What is GhostWire?

GhostWire lets people connect and communicate directly, forming a mesh network using whatever technology is available: Bluetooth, WiFi, LoRa, WebRTC, or even standard internet. It bridges different protocols (like Briar, Meshtastic, Matrix) and puts privacy and security first.

- **For non-technical readers:** Think of GhostWire as a walkie-talkie for the digital age, but smarter and more private. You can send messages, share files, and connect with others, even if the internet is blocked or down.
- **For technical readers:** GhostWire is a modular, extensible platform built in Rust and TypeScript, supporting pluggable transports, protocol adapters, and advanced security modules.

---

2. Why Mesh Networking?

- **Resilience:** Mesh networks do not rely on a single server or internet connection. If one device goes down, others keep the network alive.
- **Privacy:** No central authority means enhanced data protection and user autonomy.
- **Flexibility:** Works with many technologies, including Bluetooth, WiFi, LoRa, and more.

---

3. Real-World Scenarios

Communities with Limited Connectivity

- **Problem:** Internet blackouts and connectivity constraints.
- **Solution:** GhostWire forms a local mesh, letting communities communicate securely and privately.

Disaster Response

- **Problem:** Infrastructure is down after a natural disaster.
- **Solution:** First responders use GhostWire to coordinate rescue efforts, even without cell towers.

Rural Communities

- **Problem:** No reliable internet access.
- **Solution:** GhostWire connects villages using LoRa and WiFi, enabling messaging and information sharing.

Community Events

- **Problem:** Overloaded networks at large gatherings.
- **Solution:** Attendees use GhostWire to share updates and stay connected.

---

4. Key Features & Benefits

- **Decentralized:** No single point of failure.
- **Privacy-First:** End-to-end encryption, metadata protection, and traffic obfuscation.
- **Modular:** Add or remove transports and adapters as needed.
- **Cross-Protocol:** Bridge to Briar, Meshtastic, Matrix, and more.
- **User-Friendly:** Simple web and mobile interfaces.
- **Open Source:** Transparent, auditable, and community-driven.

---

5. How GhostWire Works (Plain & Technical)

Plain-Language Overview

- Devices connect directly to each other, forming a web (mesh).
- Messages hop from device to device until they reach their destination.

- No internet? No problem, use Bluetooth, WiFi, or LoRa.
- Everything is encrypted and private.

The Device Hierarchy

Not all devices are equal in GhostWire. The network organizes devices into tiers based on available RAM, storage, and power. A nearby laptop can take over relay duty from a phone, keeping the mesh alive when a low-power device needs to rest. This tiered approach keeps the network stable as it grows. The hierarchy works like this: phones and small devices handle sending and receiving, while laptops and servers take on heavier relay and storage tasks. As one project note puts it, "A flat mesh becomes unstable beyond a certain density. We should introduce a tiered architecture."

AI Routing

GhostWire uses artificial intelligence to decide how messages travel across the mesh. A lightweight classification model examines each message and picks the best transport and route. A larger analytical model runs periodically to update the network map. This means messages can find their way even when the network is changing constantly, and the AI models run on devices themselves, not on a remote server.

Technical Deep Dive

- **Core:** Rust backend with modular traits for transports, adapters, and security.
- **Frontend:** React/TypeScript web UI, CLI for power users.
- **Transports:** Pluggable modules for Bluetooth, WiFi, LoRa, WebRTC, TCP/IP.
- **Adapters:** Protocol bridges for Briar, Meshtastic, Matrix, and more.
- **Security:** End-to-end encryption (AES-256-GCM, X25519), Sybil defense, quotas, blacklists, traffic obfuscation.
- **Store & Forward:** Messages are cached and relayed when possible.

---

6. Visual Guide: GhostWire in Action

```
graph TD;
  User1["User A (Phone)"] -- Bluetooth --> Node1["GhostWire Node"]
  User2["User B (Laptop)"] -- WiFi --> Node1
  Node1 -- LoRa --> Node2["Remote Node"]
  Node2 -- WebRTC --> User3["User C (Browser)"]
  Node1 -- Matrix Adapter (implemented) --> Matrix["Matrix Network"]
  Node1 -- Briar Adapter (planned) --> Briar["Briar Network"]
```

---

7. Best Practices & Anti-Patterns

Best Practices

- Use multiple transports for resilience.
- Keep software updated for latest security patches.
- Use strong, unique passwords for device access.

- Educate users about privacy and security features.

Anti-Patterns

- Relying on a single transport (e.g., only WiFi).
- Disabling encryption or security modules.
- Ignoring software updates.

---

8. Project Status (September 2026)

GhostWire is an active, growing project. As of September 2026, the codebase contains approximately 74,000 lines of Rust across 181 files. The project has applied to the Open Technology Fund for $200,000 over 24 months, structured as 12 milestones across 8 phases.

Transports Working Today

Five transports are functional: WiFi Direct, LoRa, stealth TCP, WebRTC, and Reticulum. Bluetooth LE support is in progress. Traffic Armor provides camouflage and auto-switching when a transport is blocked, using Thompson Sampling for energy-aware transport selection (energy weights: BLE 0.30, LoRa 0.35, Reticulum 0.25, WiFi 0.05, TCP/QUIC 0).

Protocol Bridges

- **Matrix:** Implemented behind the `matrix-bridge` feature flag. Basic form is working.
- **Meshtastic:** Implemented with erasure coding (3 data + 2 parity shards, any 3 of 5 reconstruct). Works around the 242-byte LoRa frame limit.
- **Briar:** Planned. Not yet implemented.
- **Reticulum:** Planned/experimental, behind a feature flag.
- **Tor:** Not implemented.

External Security Audit

An external security audit is planned but has not yet been conducted. Candidate auditors include Cure53, NCC Group, and Least Authority.

New Chapters

Three new chapters expand the Bible: Chapter 15 covers device hierarchy and power-aware scaling, Chapter 16 covers AI routing and on-device intelligence, and Chapter 17 covers the people and communities GhostWire serves.

---

9. Frequently Asked Questions

Q: Is GhostWire legal to use? A: In most countries, yes, but always check local laws, especially regarding encryption and radio use.

Q: Can I use GhostWire without the internet? A: Yes. That is one of its main features.

Q: How secure is GhostWire? A: It uses strong encryption and privacy techniques, but no system is 100% secure. Follow best practices.

Q: Can I contribute? A: Absolutely. See the Contributing chapter.

---

10. Further Reading & Resources

- GhostWire GitHub
- Mesh Networking 101
- Briar Project
- Meshtastic
- Matrix Protocol
- LoRa Technology
- Device Hierarchy & Power-Aware Scaling
- AI Routing & On-Device Intelligence
- Who GhostWire Serves

Appendix: Glossary

- **Mesh Network:** A network where each device relays data for others.
- **Transport:** The method used to send messages (Bluetooth, WiFi, etc.).
- **Protocol Adapter:** Software that bridges GhostWire to other networks.
- **Node:** Any device running GhostWire.
- **Sybil Attack:** When one entity pretends to be many nodes.
- **Quota:** Limit on messages/actions to prevent abuse.

End of Chapter

Getting Started with GhostWire

Table of Contents

1. Welcome & Audience
2. What You Can Do with GhostWire
3. Quick Start (No Coding)
4. Full Setup (Technical)
5. Choosing Your Role and Transport
6. Joining & Creating a Mesh
7. Using the Web UI & CLI
8. Troubleshooting & Support
9. Real-World Onboarding Scenarios
10. Best Practices for New Users
11. Further Reading & Resources

---

1. Welcome & Audience

GhostWire is for everyone, community organizers, first responders, privacy advocates, rural users, and developers. This guide walks you through your first steps, no matter your background.

---

2. What You Can Do with GhostWire

- **Send messages** to friends, family, or colleagues, even if the internet is down.
- **Join a local mesh** at an event, protest, or in your neighborhood.
- **Bridge to other networks** (like Briar or Meshtastic) for wider reach.
- **Contribute to open-source** and help build the future of secure, decentralized communication.

---

3. Quick Start (No Coding)

Option 1: Web Demo

1. Visit the GhostWire demo site (if available).
2. Follow the on-screen instructions to join a mesh and send your first message.

Option 2: Mobile/Desktop App

1. Download the GhostWire app for your platform (links on the project website).
2. Install and open the app.
3. Choose your preferred transport (WiFi Direct, LoRa, WebRTC).
4. Join or create a local mesh.
5. Start messaging!

---

4. Full Setup (Technical)

Prerequisites

- **Rust** (for backend/CLI): Install Rust
- **Node.js & npm** (for web UI): Install Node.js
- **LoRa hardware** (optional, for long-range)

Step-by-Step

1. **Clone the repo:** `sh  git clone https://github.com/phantomojo/GhostWire-secure-mesh-communica`
   `cd GhostWire-secure-mesh-communication/ghostwire`
2. **Build the backend:** `sh  cargo build --release`
3. **Run the backend:** `sh  cargo run --release`
4. **Start the web UI:** `sh  cd ../../webui  npm install  npm run dev`
5. **Access the UI:** Open your browser to http://localhost:3000

---

5. Choosing Your Role and Transport

GhostWire runs on a range of devices, and your hardware determines what role you can play in the mesh.

Device	Memory	Max Peers	Storage	Typical Role
Phone	<=6GB	20	1GB	Leaf (best for messaging)
Tablet	<=8GB	30	2GB	Leaf or light Relay
Raspberry Pi	<=4GB	15	500MB	Relay or Sensor
Laptop	<=16GB	50	5GB	Good Relay
Server	>16GB	100+	50GB+	Best Relay, Replicator

Which role fits you? - **Phone users** are best suited as leaf devices. You get full messaging, file sharing, and group chat, but your device is not expected to relay traffic for others. - **Laptop and Raspberry Pi users** make good relays. You can forward messages for nearby nodes and extend mesh range. - **Always-on machines (server-class)** are the best relays. They provide stable, high-capacity connectivity and can act as Replicators, storing and forwarding messages for the whole mesh.

Battery States and Privacy

Your device's battery level affects which privacy tier GhostWire assigns you: - **Charging (any level):** Unrestricted. You get the full privacy tier for your device class. - **20-50% battery:** Cover traffic and some advanced features are reduced to conserve energy. - **20% or below:** Drops to privacy tier L0 or L1. Non-essential traffic is suppressed, and the device operates in a minimal-footprint mode.

Pairing with QR Codes

To join a mesh quickly, scan a QR code displayed in the UI or shared by a mesh organizer. QR pairing establishes the initial connection and encrypts the session key, so you do not need to type a mesh ID manually.

---

6. Joining & Creating a Mesh

- **Auto-Discovery:** GhostWire finds nearby nodes using WiFi Direct, LoRa, or WebRTC.
- **Manual Join:** Enter a mesh ID or scan a QR code to join a specific group.
- **Creating a Mesh:** Click "Create Mesh" in the UI, set a name and (optional) password, and invite others.

---

7. Using the Web UI & CLI

Web UI

- **Dashboard:** See connected nodes, active transports, and recent messages.
- **Chat:** Send/receive messages, share files, create groups.

- **Settings:** Choose transports, manage keys, set quotas, enable/disable adapters.
- **Visuals:** `mermaid graph TD; User["You"] -->|Web UI| GhostWire["GhostWire Node"] GhostWire -->|WiFi Direct/LoRa/WebRTC| Mesh["Mesh Network"]`

CLI (for Power Users)

- **Start a node:** `sh ghostwire-cli start --transport wifi --mesh mymesh`
- **Send a message:** `sh ghostwire-cli send --to bob --message "Hello, Bob!"`
- **List nodes:** `sh ghostwire-cli nodes`

---

8. Troubleshooting & Support

Problem	Solution
Can't find other nodes	Check transport settings, try another method
Messages not sending	Ensure at least one transport is active
Web UI won't load	Check backend is running, try npm install
LoRa not working	Check hardware, drivers, and permissions
Security warning	Ensure you're using the latest version

- **Logs:** Check backend logs for errors.
- **Community:** Ask for help on GitHub or project chat.

---

9. Real-World Onboarding Scenarios

Community Event

- **Goal:** Set up a mesh for a festival or protest.
- **Steps:**
  1. Organizers install GhostWire on phones/laptops.
  2. Create a mesh and share the QR code.
  3. Attendees join and start messaging.

Disaster Response

- **Goal:** Connect first responders in a blackout.
- **Steps:**
  1. Deploy LoRa nodes at key locations.
  2. Responders join the mesh via mobile or CLI.
  3. Use store-and-forward to relay messages.

Rural Village

- **Goal:** Connect homes with no internet.
- **Steps:**
  1. Install GhostWire on home computers.
  2. Use WiFi Direct or LoRa to form a mesh.

3. Share news, alerts, and messages.

---

10. Best Practices for New Users

- Always use the latest version for security.
- Try multiple transports for best coverage.
- Use strong passwords for mesh access.
- Learn about privacy features in the Security chapter.
- Join the community for support and updates.

---

11. Further Reading & Resources

- GhostWire GitHub
- Mesh Networking 101
- LoRa Setup Guide
- CLI Reference
- Security & Privacy

End of Chapter

Architecture Deep Dive

Table of Contents

1. Overview & Philosophy
2. System Diagram & Visuals
3. Core Components (Plain & Technical)
4. Device Hierarchy and Supernodes
5. Transports Layer
6. Protocol Adapters Layer
7. AI Routing Layer
8. Security & Trust Layer
9. Store & Forward / Federation
10. Data Flow: Message Lifecycle
11. Deployment Blueprints
12. Best Practices & Anti-Patterns
13. Glossary & Reference

---

1. Overview & Philosophy

GhostWire is built for modularity, security, and real-world flexibility. The architecture is designed to: - Support multiple transports and protocols - Enable privacy and resilience by default - Allow easy extension and adaptation for new use cases

---

2. System Diagram & Visuals

```
graph TD;
  User["User"] -->|Web UI| WebFrontend["React/Tailwind Web UI"]
  User -->|CLI| CLI["Rust CLI"]
  WebFrontend -->|REST/WebSocket| Backend["Rust Backend"]
  CLI --> Backend
  Backend -->|Transports| Transports["WiFi Direct, LoRa, stealth TCP, WebRTC, Reticulum"]
  Backend -->|Adapters| Adapters["Matrix (implemented), Meshtastic (implemented), Briar (planne
  Backend --> AI["AI Routing Layer"]
  Backend --> Security["Security Modules"]
  Backend --> Store["Store & Forward"]
  Security -->|Sybil Defense, Quotas, Blacklists| Backend
  Store -->|Federation| OtherMesh["Other GhostWire Mesh"]
```

---

3. Core Components (Plain & Technical)

Plain-Language

- **Web UI:** The dashboard you use in your browser.
- **CLI:** Command-line tool for advanced users.
- **Backend:** The "brain" that connects everything, runs on your device.
- **Transports:** The "roads" messages travel on (WiFi Direct, LoRa, stealth TCP, WebRTC, Reticulum).
- **Adapters:** "Translators" that let GhostWire talk to other networks.
- **Security Modules:** Keep your messages private and your network safe.
- **Store & Forward:** Lets messages wait and be delivered later if needed.

Technical

- **Rust Backend:** Implements core traits: `Transport`, `ProtocolAdapter`, `KeyManager`, `QuotaEnforcer`, etc.
- **Frontend:** React/TypeScript, communicates via REST/WebSocket APIs.
- **Transports:** Each is a Rust module implementing the `Transport` trait, can be enabled/disabled at runtime.
- **Adapters:** Rust modules implementing `ProtocolAdapter`, handle translation, deduplication, and relay.
- **Security:** Modular, pluggable, with Sybil defense, quotas, blacklists, traffic obfuscation, and more.
- **Store & Forward:** Message cache, relay, and federation logic.

4. Device Hierarchy and Supernodes

A flat mesh becomes unstable beyond a certain density. We introduce a tiered architecture so that devices with more resources can serve as relays and anchors, while resource-constrained devices focus on messaging.

Device Classes

Class	Memory	Max Peers	Storage	Role
Phone	<=6GB	20	1GB	Leaf
Tablet	<=8GB	30	2GB	Leaf or light Relay
Raspberry Pi	<=4GB	15	500MB	Relay or Sensor
Laptop	<=16GB	50	5GB	Relay
Server	>16GB	100+	50GB+	Full, Replicator

Roles

- **Full Node:** Maintains the complete mesh state, participates in routing and relay. Typically a server or always-on laptop.
- **Relay:** Forwards messages between nodes. Can be a Raspberry Pi, laptop, or tablet with sufficient uptime and bandwidth.
- **Leaf:** A messaging endpoint that does not relay traffic for others. Phones and lightweight tablets typically operate as leaves.
- **Sensor:** A device that generates or consumes data but does not relay. Examples include environmental sensors or battery-constrained devices.
- **Replicator:** A full node that stores and replicates messages for the mesh, ensuring availability even when original senders are offline. Typically a server-class machine.

Supernode Election

Dynamically elect nodes with high uptime, stable IP addresses, and significant bandwidth to act as "Relay Hubs." These supernodes coordinate routing tables and serve as anchor points for the mesh. Election is based on observable metrics: uptime history, bandwidth availability, and peer count. A node does not need to be a server to become a supernode; a Raspberry Pi with stable power and connectivity can qualify.

Trust and Relay Authority

Relays carry no trust authority. A relay forwards messages but cannot validate, modify, or censor the content it carries. Trust is established end-to-end between the original sender and the final recipient through cryptographic signatures. This separation ensures that even a compromised relay cannot impersonate a user or read message contents.

Hardware Ladder and ESP32 Gateway Bridge

The hardware ladder runs from a $20 ESP32 through a $100 Raspberry Pi gateway to server-class machines. The ESP32 gateway-bridged architecture creates a mini-mesh locally and bridges it to a

WiFi uplink, allowing low-cost sensors and devices to participate in the mesh through a gateway node. This is ideal for rural deployments and disaster scenarios where dedicated hardware is scarce.

Mailbox Pattern for Leaf Devices

Leaf devices that cannot maintain persistent connections use a mailbox pattern. Messages destined for a leaf are cached at the leaf's last-known relay node. When the leaf reconnects, it pulls all queued messages in batch. This decouples availability from connectivity.

---

5. Transports Layer

- **Working today:** WiFi Direct, LoRa, stealth TCP, WebRTC, Reticulum (feature-flagged)
- **In progress:** Bluetooth LE
- **Pluggable:** Add new transports by implementing the `Transport` trait.
- **Runtime Selection:** Enable/disable transports via config or UI.
- **Visual:** `mermaid graph LR; Backend --> WiFiDirect["WiFi Direct"] Backend --> LoRa["LoRa"] Backend --> StealthTCP["stealth TCP"] Backend --> WebRTC["WebRTC"] Backend --> Reticulum["Reticulum"]`
- **Best Practice:** Use multiple transports for resilience.

---

6. Protocol Adapters Layer

- **Purpose:** Bridge GhostWire to other networks (Matrix, Meshtastic, Briar, etc.)
- **How:** Implement the `ProtocolAdapter` trait.
- **Features:** Message translation, deduplication, relay, group chat, file sharing.
- **Visual:** `mermaid graph LR; Backend --> MatrixAdapter["Matrix Adapter (implemented)"] Backend --> MeshtasticAdapter["Meshtastic Adapter (implemented)"] Backend --> BriarAdapter["Briar Adapter (planned)"] MatrixAdapter --> MatrixNetwork MeshtasticAdapter --> MeshtasticNetwork BriarAdapter --> BriarNetwork`

---

7. AI Routing Layer

The AI Routing Layer applies machine learning to decide how messages are classified, routed, and prioritized across the mesh. It operates across four distinct layers, each with a different scope and cadence.

Layer 1: LightGBM Per-Message Classification

Every message passes through a LightGBM classifier that assigns it a priority category. This inference runs in approximately 77 microseconds on a Raspberry Pi, so it adds negligible latency. Features include message size, destination type, transport metrics, and battery level. The classifier decides whether a message is routine, priority, or emergency, which affects routing decisions and quota enforcement.

Layer 2: Gemma 4 Analytical Pass

Every 30 seconds, a Gemma 4 model runs an analytical pass over recent mesh activity. This pass identifies traffic patterns, anomalies, and potential congestion points. The output feeds into routing table adjustments and supernode election decisions. This is a periodic batch process, not a per-message classifier.

Layer 3: GNN Graph Router

A Graph Neural Network models the mesh as a dynamic graph where nodes are devices and edges are transport links with associated latency, reliability, and energy cost. The GNN computes optimal routing paths in real time, adapting to topology changes. This layer handles the heavy lifting of pathfinding across a volatile mesh.

Layer 4: PRoPHET Delay-Tolerant Routing

For partitions and disconnected scenarios, PRoPHET (Probabilistic Routing Protocol Using History of Encounters and Transitivity) provides delay-tolerant message forwarding. Messages are carried by nodes that are likely to encounter the destination in the future. This is essential for disaster response and off-grid scenarios where continuous connectivity cannot be guaranteed.

Model Ladder

Models are deployed across the hardware ladder based on capability: - **E2B (Edge-to-Base):** Runs on edge devices (phones, Raspberry Pi) for lightweight classification. - **E4B (Edge-to-Cloud):** Runs on relay laptops for analytical passes and local graph inference. - **26B MoE (Mixture of Experts) Coordinator:** Runs on a coordinator node to aggregate routing decisions across the mesh. - **31B Server Model:** Runs on server-class machines for full analytical depth and training.

Telemetry and Model Distribution

Telemetry flows up (opt-in only) so that model training data is collected with explicit user consent. Model files flow down via DHT hash verification and P2P sync, not federated learning. Each node verifies model integrity through content hashes distributed over the DHT before loading a new model version.

Store-and-Forward DTN Integration

The AI Routing Layer integrates with store-and-forward DTN protocols (PRoPHET, Spray-and-Wait) to make intelligent decisions about when to hold messages versus when to forward them. In partitioned networks, the AI predicts which relay nodes are most likely to reach the destination and queues messages accordingly.

---

8. Security & Trust Layer

- **Modules:**
  - SybilDefense
  - QuotaEnforcer
  - BlacklistManager

  - TrafficObfuscator
  - KeyManager
- **Features:**
  - End-to-end encryption (AES-256-GCM, X25519)
  - Perfect forward secrecy
  - Ephemeral keys, key rotation
  - Quotas and rate limiting
  - Blacklisting and abuse prevention
  - Traffic obfuscation and anti-analysis
  - ML-KEM-768 post-quantum key encapsulation (890us)
  - SPHINX SURBs and cover traffic for metadata protection
  - Privacy tiers: L0 (minimal), L1 (standard), L2 (enhanced), L3 (maximum)
- **Privacy Tiers by Device:**
  - Server: L3 (maximum privacy)
  - Laptop and Tablet: L2
  - Phone and Raspberry Pi: L1 (drops one tier when battery is 20% or below)
- **Visual:** `mermaid graph TD; Backend --> Security["Security Modules"] Security --> SybilDefense Security --> QuotaEnforcer Security --> BlacklistManager Security --> TrafficObfuscator Security --> KeyManager Security --> Privacy["Privacy Tiers L0-L3"] Security --> SPHINX["SPHINX SURBs + Cover Traffic"]`

---

9. Store & Forward / Federation

- **Store & Forward:** Messages are cached and relayed when possible. DTN protocols (PRoPHET, Spray-and-Wait) handle disconnected scenarios.
- **Federation:** Meshes can connect to each other for wider reach.
- **Visual:** `mermaid graph TD; NodeA --> Store Store --> NodeB Store --> Federation["Other Mesh"]`

---

10. Data Flow: Message Lifecycle

1. **User sends message via UI/CLI**
2. **Backend encrypts and signs message**
3. **AI Routing Layer classifies and selects route**
4. **Message routed via best available transport(s)**
5. **Adapters translate if needed**
6. **Security modules enforce quotas, check blacklists**
7. **Message hops node-to-node (store & forward as needed)**
8. **Recipient decrypts and reads message**

---

11. Deployment Blueprints

Home/Personal Mesh

- Single device or small group, WiFi Direct/Bluetooth
- Simple setup, auto-discovery
- Phone as leaf, laptop as relay

Community/Neighborhood Mesh

- Dozens of nodes, mix of WiFi Direct, LoRa, WebRTC
- Some nodes act as relays and supernodes
- Store & forward for offline delivery
- Raspberry Pi gateways bridge ESP32 sensors

Disaster/Field Deployment

- LoRa nodes at key locations
- Battery/solar-powered relays
- Store & forward, federation with other meshes
- PRoPHET routing for partitioned scenarios
- Privacy tier drops to L0/L1 on low battery

Enterprise/Federated Mesh

- Multiple sites, federation, quotas, advanced security
- Integration with existing systems via adapters
- Server-class nodes as Replicators and supernodes
- Matrix bridge for external reach

---

12. Best Practices & Anti-Patterns

Best Practices

- Use multiple transports for resilience
- Enable all relevant security modules
- Regularly update software
- Monitor mesh health and logs
- Educate users on privacy and security
- Use the AI Routing Layer to optimize message priority
- Verify model integrity via DHT hashes before loading

Anti-Patterns

- Relying on a single transport
- Disabling security features
- Ignoring updates or logs
- Running relays without adequate uptime guarantees
- Assuming relays have trust authority (they do not)

---

13. Glossary & Reference

- **Node:** Any device running GhostWire
- **Transport:** WiFi Direct, LoRa, stealth TCP, WebRTC, Reticulum
- **Adapter:** Bridge to other protocols
- **Supernode:** A dynamically elected relay hub with high uptime and bandwidth
- **Sybil Attack:** One entity pretends to be many nodes
- **Quota:** Limit on messages/actions
- **Federation:** Connecting multiple meshes
- **DTN:** Delay-Tolerant Networking, for disconnected scenarios
- **SPHINX:** Source-routed onion protocol for metadata protection
- **Privacy Tier:** L0-L3, controlling the level of traffic obfuscation and cover traffic

---

End of Chapter

Transports & Protocols

Table of Contents

1. What is a Transport?
2. Supported & Planned Transports
3. Traffic Armor and Auto-Switching
4. Transport Selection: Thompson Sampling
5. How Transports Work (Plain & Technical)
6. Which Transport When
7. Real-World Use Cases
8. Configuration & Code Examples
9. Visual Guide: Transport Topologies
10. Best Practices & Anti-Patterns
11. Troubleshooting & FAQ
12. Further Reading & Resources

---

1. What is a Transport?

- **Plain:** A transport is the "road" your messages travel on: WiFi Direct, LoRa, stealth TCP, WebRTC, and more.
- **Technical:** In GhostWire, each transport is a pluggable module implementing the `Transport` trait, allowing for runtime or compile-time enable/disable.

---

2. Supported & Planned Transports

Transport	Status	Use Case / Notes
WiFi Direct	Working	Local mesh, high bandwidth, urban/rural
LoRa	Working	Long-range, low-power, rural, disaster, off-grid
stealth TCP	Working	Evades DPI, auto-escalates when blocked
WebRTC	Working	Browser-to-browser, disguises as video call traffic
Reticulum	Working (feature-flagged)	Encapsulated transport, enable with `reticulum = true`
Bluetooth LE	In progress	Short-range, mobile-to-mobile, coming soon

---

3. Traffic Armor and Auto-Switching

GhostWire's Traffic Armor makes your mesh traffic look like ordinary web traffic so it is harder to identify, block, or throttle.

- **WebRTC Disguise:** WebRTC traffic is shaped to look like a Discord-style voice or video call. Deep packet inspection cannot easily distinguish it from legitimate real-time communication.
- **stealth TCP:** Evades DPI (Deep Packet Inspection) by obfuscating packet headers and payload patterns. When a network blocks stealth TCP, the transport auto-escalates: it falls back to WebRTC or Reticulum, which use different obfuscation strategies.
- **Auto-Switching:** If one transport is blocked or degraded, GhostWire automatically tries alternatives. The selection is not random; it is guided by the Thompson Sampling algorithm described below.

---

4. Transport Selection: Thompson Sampling

GhostWire uses Thompson Sampling, a bandit algorithm, to balance reliability, latency, and energy efficiency when choosing which transport to use for each message. Each transport is modeled as a slot machine with a probability distribution over its expected quality. The algorithm explores transports that are uncertain and exploits those that have proven reliable.

Energy-efficiency weights guide the exploration-exploitation tradeoff:

Transport	Energy Efficiency Weight
Bluetooth LE	0.30
LoRa	0.35
Reticulum	0.25
WiFi Direct	0.05
TCP/QUIC	0.00

Higher weights mean the transport is preferred when energy is the dominant concern. On battery-powered devices, the algorithm favors LoRa and Bluetooth LE. On plugged-in devices, WiFi Direct and WebRTC get higher priority because they offer lower latency and higher bandwidth.

---

5. How Transports Work (Plain & Technical)

Plain-Language

- Devices use whatever "roads" are available to connect: WiFi Direct for local mesh, LoRa for long distance, WebRTC for browser-based communication.
- GhostWire automatically picks the best available transport, or you can choose manually.

Technical

- Each transport implements the `Transport` trait in Rust: `rust pub trait Transport { fn send(&self, msg: Message) -> Result<(), TransportError>; fn receive(&self) -> Option<Message>; fn is_available(&self) -> bool; // ... }`
- Transports can be enabled/disabled at runtime via config or UI.
- Multiple transports can be active at once for resilience.

---

6. Which Transport When

Scenario	Recommended Transport(s)	Why
Indoor messaging, low latency	WiFi Direct	High bandwidth, low latency, no external hardware needed
Rural area, no internet	LoRa	Long-range, low-power, works without infrastructure
Censored or monitored network	stealth TCP, WebRTC	Evades DPI, disguises as ordinary traffic
Browser-based peer-to-peer	WebRTC	No plugins, NAT traversal built in

Scenario	Recommended Transport(s)	Why
Maximum anonymity, obfuscation	Reticulum	Encapsulated, feature-flagged for advanced users
Mobile, battery-constrained	Bluetooth LE (when available)	Very low energy, short range
Mixed environment	Multiple transports active	Thompson Sampling picks the best option per message

---

7. Real-World Use Cases

Disaster Response

- **Scenario:** Power and cell towers are down after a hurricane.
- **Solution:** LoRa nodes relay messages across miles; WiFi Direct and WebRTC fill in gaps.

Urban Mesh

- **Scenario:** Protesters need secure, local communication.
- **Solution:** Phones use WiFi Direct and WebRTC to form a dense, resilient mesh. stealth TCP evades network monitoring.

Rural Connectivity

- **Scenario:** Villages with no internet need to share news.
- **Solution:** LoRa radios connect homes and farms over long distances.

Connectivity in Constrained Environments

- **Scenario:** Authorities restrict internet access or monitor traffic.
- **Solution:** stealth TCP and WebRTC provide obfuscated, hard-to-block channels. Traffic Armor disguises the traffic as ordinary video-call or web traffic.

---

8. Configuration & Code Examples

Enabling/Disabling Transports (Config)

```
[transports]
bluetooth = false
wifi = true
```

```
lora = true
webrtc = true
tcpip = true
stealth_tcp = true
# reticulum = true  # Uncomment to enable Reticulum (feature-flagged)
```

Using Transports in Code

```
let wifi = WifiDirectTransport::new();
let lora = LoRaTransport::new();
backend.add_transport(Box::new(wifi));
backend.add_transport(Box::new(lora));
```

Web UI Example

- Go to Settings > Transports
- Toggle available transports on/off
- See real-time status and diagnostics

---

9. Visual Guide: Transport Topologies

```
graph TD;
  Phone1["Phone"] -- WiFi Direct --> Phone2["Phone"]
  Phone2 -- LoRa --> Laptop["Laptop"]
  Laptop -- LoRa --> Relay["LoRa Relay"]
  Relay -- LoRa --> Village["Remote Village Node"]
  Laptop -- WebRTC --> Browser["Browser"]
  Laptop -- stealth TCP --> Internet["Internet Node"]
  Laptop -- Reticulum --> Mesh["Reticulum Mesh"]
```

---

10. Best Practices & Anti-Patterns

Best Practices

- Enable multiple transports for best coverage.
- Test transport availability before deployment.
- Use LoRa for long-range, low-power needs.
- Use WebRTC or stealth TCP for connectivity in constrained environments.
- Monitor transport health in the UI.
- Let Thompson Sampling manage transport selection when you are unsure.

Anti-Patterns

- Relying on a single transport.
- Disabling security features on transports.
- Ignoring hardware compatibility.
- Forcing a transport when Thompson Sampling would choose a better option.

11. Troubleshooting & FAQ

Problem	Solution
Can't connect via Bluetooth LE	Check permissions, try WiFi Direct or LoRa
LoRa not working	Check hardware, drivers, and range
WebRTC fails	Check firewall/NAT, try TCP/IP or stealth TCP fallback
stealth TCP blocked	Auto-escalation should switch to WebRTC or Reticulum; check if feature flag is enabled
Slow performance	Use higher-bandwidth transport if possible
Security warning	Ensure encryption is enabled on all transports

---

12. Further Reading & Resources

- LoRa Alliance
- Bluetooth Mesh
- WebRTC
- Mesh Networking 101
- GhostWire Developer Guide

End of Chapter

Security & Privacy

Table of Contents

1. Executive Summary
2. Security Foundations (Plain & Technical)
3. Threat Models & Real-World Risks
4. Security Architecture & Modules
5. Encryption & Key Management
6. Sybil Defense & Trust
7. Quotas, Blacklists, and Abuse Prevention
8. Traffic Obfuscation & Anti-Analysis
9. Disaster & Infrastructure Failure Scenarios
10. Best Practices & Anti-Patterns
11. Actionable Security Checklists
12. Visuals: Security Layers & Flows
13. FAQ & Troubleshooting
14. Further Reading & Resources

1. Executive Summary

GhostWire is built with security and privacy as core principles. This chapter explains how GhostWire protects users, what threats it defends against, and how to use its security features, whether you are a non-technical user or a security engineer.

---

2. Security Foundations (Plain & Technical)

- **Plain:** GhostWire keeps your messages private and your identity safe, even if someone tries to spy or block you.
- **Technical:** End-to-end encryption (AES-256-GCM, X25519), perfect forward secrecy, ephemeral keys, key rotation, secure storage, and post-quantum crypto (planned).

---

3. Threat Models & Real-World Risks

- **Connectivity Loss:** Governments, ISPs, or natural disasters causing communication blackouts.
- **Surveillance:** Adversaries trying to read or analyze messages.
- **Sybil Attacks:** Fake nodes trying to disrupt or spy on the mesh.
- **Denial of Service:** Flooding the network to disrupt communication.
- **Traffic Analysis:** Inferring who is talking to whom, even if messages are encrypted.
- **Device Seizure:** Physical access to a device running GhostWire.

---

4. Security Architecture & Modules

- **SybilDefense:** Prevents fake nodes from overwhelming the mesh.
- **QuotaEnforcer:** Limits message rates to prevent spam/DoS.
- **BlacklistManager:** Blocks known abusers or compromised nodes.
- **TrafficObfuscator:** Makes traffic patterns harder to analyze.
- **KeyManager:** Handles encryption keys, rotation, and secure storage.

---

5. Encryption & Key Management

- **End-to-end encryption:** All messages are encrypted from sender to recipient.
- **Key exchange:** X25519 for secure, ephemeral key exchange.
- **Key rotation:** Regularly rotates keys for forward secrecy.
- **Secure storage:** Keys are stored encrypted on disk.
- **Post-quantum:** Research and planning for future upgrades.

---

6. Modern Crypto & Identity Stack

GhostWire deploys a layered cryptographic and identity system designed for both current threats and future risks.

Post-Quantum Key Encapsulation

- **ML-KEM-768:** GhostWire integrates ML-KEM-768 (Module-Lattice-Based Key-Encapsulation Mechanism at the 768-bit security level) for post-quantum key establishment. On-device key encapsulation completes in approximately 890 microseconds, making it practical for mobile and embedded hardware.
- **Hybrid mode:** ML-KEM-768 is combined with X25519 during the transition period, so a session retains security even if one algorithm is later broken.

Symmetric Encryption

- **AES-256-GCM:** The default cipher for small to medium payloads. Provides authenticated encryption with associated data (AEAD), ensuring both confidentiality and integrity.
- **XChaCha20-Poly1305:** Used for large payloads where AES-NI hardware acceleration is unavailable. XChaCha20 uses a 192-bit nonce, eliminating nonce-reuse risks that plague standard ChaCha20. It outperforms AES-256-GCM on devices without AES hardware support, such as Raspberry Pi and older phones.

Key Exchange and Forward Secrecy

- **X25519:** Elliptic-curve Diffie-Hellman key exchange on the Curve25519 group. Provides fast, constant-time key agreement.
- **Perfect Forward Secrecy (PFS):** Every session generates ephemeral key pairs. Compromise of a long-term key does not expose past session traffic. Keys are rotated after each session and at defined intervals during active conversations.

Duress PIN and Coercion Resistance

- **Duress PIN:** Users can configure a secondary PIN that, when entered under coercion, unlocks a plausible-deniability mode. The device presents a harmless or empty view of the mesh while the real identity and keys remain cryptographically hidden.
- **Plausible deniability under seizure:** If a device is seized, the duress PIN mode ensures that an adversary cannot distinguish whether real data exists on the device. Encrypted volumes and key material remain inaccessible without the primary credentials.

Identity: DIDs, Shamir Secret Sharing, Dead Man's Switch

- **Decentralized Identifiers (DIDs):** GhostWire assigns each node a DID that is not tied to any central authority. DIDs enable verifiable, self-sovereign identity across meshes without relying on a directory server.
- **Shamir Secret Sharing:** Recovery keys are split into shares distributed among trusted contacts. A threshold number of shares is required to reconstruct the master key, preventing single-point failure and single-point compromise.

- **Dead Man's Switch:** If a node goes silent beyond a configured timeout, the dead man's switch automatically triggers. It can wipe local keys, notify trusted contacts, or publish a revocation certificate, depending on configuration.
- **Device Pairing:** New devices pair via a secure out-of-band channel (QR code scan, near-field broadcast, or in-person verification). Pairing establishes a shared secret without exposing it to network observers.
- **KERI (Key Event Receipt Infrastructure):** Planned for future integration. KERI provides a fully decentralized, append-only log of key events, enabling trust anchoring without certificate authorities.

Privacy Tiers (L0-L3)

GhostWire enforces privacy levels that adapt to device capabilities and battery state:

Tier	Device Type	Metadata Exposure	Cover Traffic
L3	Server	Minimal	Full
L2	Laptop, Tablet	Reduced	Partial
L1	Phone, Raspberry Pi	Basic	Minimal
L0	Any device (battery critical)	Maximum	None

- **Server nodes** operate at L3 with full cover traffic and minimal metadata leakage.
- **Laptops and tablets** operate at L2 with reduced metadata and partial cover traffic.
- **Phones and Raspberry Pi** operate at L1 with basic protections.
- **When battery drops below 20%,** the device automatically drops one privacy tier to conserve energy.

SPHINX Unlinkable Replies and Cover Traffic

- **SPHINX (Scalable Hybrid Incremental N-way Exchange):** GhostWire uses SPHINX-style onion routing for replies. Each message is wrapped in multiple layers of encryption so that intermediate nodes can only decrypt the layer intended for them. No single node knows both the sender and the ultimate recipient.
- **Unlinkable Replies (SURBs):** Single-Use Reply Blocks allow a recipient to reply without revealing their identity to the sender or any intermediary. Each SURB is consumed after one use.
- **Cover traffic:** Nodes generate dummy packets at regular intervals to mask the timing and volume of real messages. This defeats traffic analysis that relies on observing communication patterns.

---

7. Sybil Defense & Trust

- **Proof-of-Work/Stake:** Optional modules to make Sybil attacks expensive.
- **Reputation:** Nodes can build trust over time.
- **Manual approval:** Option for closed/curated meshes.

---

8. Quotas, Blacklists, and Abuse Prevention

- **Quotas:** Rate limits on messages, connections, and actions.
- **Blacklists:** Block known abusers or compromised nodes.
- **Automated & manual controls:** Admins can adjust settings in real time.

---

9. Traffic Obfuscation & Anti-Analysis

- **Padding:** Adds random data to messages to hide true size.
- **Timing obfuscation:** Randomizes message timing to prevent correlation.
- **Stealth transports:** Use WebRTC, Stealth TCP, or other obfuscated channels.
- **Traffic Armor:** WebRTC traffic is disguised to look like a video call. Stealth TCP automatically escalates to alternative protocols when blocked.

---

10. Disaster & Infrastructure Failure Scenarios

- **Disaster mode:** Store-and-forward, minimal metadata, offline queuing.
- **Connectivity resilience:** Privacy-enhanced transports, traffic obfuscation, rapid key rotation.

---

11. Best Practices & Anti-Patterns

Best Practices

- Always use the latest version.
- Enable all security modules.
- Use strong passwords and device security.
- Educate users about privacy features. ### Anti-Patterns
- Disabling encryption or security modules.
- Using default passwords.
- Ignoring updates or logs.

---

12. Actionable Security Checklists

☐ Update software regularly
☐ Enable all security modules
☐ Use strong, unique passwords
☐ Monitor mesh health and logs
☐ Educate users on privacy and security
☐ Verify ML-KEM-768 is enabled on supported nodes
☐ Confirm perfect forward secrecy is active on all sessions
☐ Test duress PIN mode before deployment
☐ Configure dead man's switch timeout
☐ Distribute Shamir secret sharing shares to trusted contacts

---

13. Visuals: Security Layers & Flows

```
graph TD;
  User["User"] --> UI["Web UI/CLI"]
  UI --> Backend["Backend"]
  Backend --> Security["Security Modules"]
  Security --> KeyManager
  Security --> SybilDefense
  Security --> QuotaEnforcer
  Security --> BlacklistManager
  Security --> TrafficObfuscator
  Security --> MLKEM["ML-KEM-768"]
  Security --> XChaCha["XChaCha20-Poly1305"]
  Security --> X25519["X25519 + PFS"]
  Security --> SPHINX["SPHINX SURBs"]
  Security --> CoverTraffic["Cover Traffic"]
```

---

14. FAQ & Troubleshooting

- **Q: How do I know my messages are secure?**
  - All messages are end-to-end encrypted by default.
- **Q: What if my device is seized?**
  - Keys are encrypted on disk; use device encryption for extra safety. Enable duress PIN mode for plausible deniability.
- **Q: Can I disable security features?**
  - Not recommended; only for advanced users in test environments.
- **Q: What is ML-KEM-768?**
  - A post-quantum key encapsulation mechanism that protects against future quantum computer attacks. It completes in 890 microseconds on-device.
- **Q: How does the duress PIN work?**
  - Entering the duress PIN activates a plausible-deniability mode that hides real data from coercive actors.

---

15. External Audit Status

An external security audit has NOT been performed. GhostWire's current protections are self-reviewed engineering controls. The development team has conducted internal reviews of the cryptographic implementations, module interactions, and threat model coverage.

External audit is PLANNED. Candidate auditing firms include Cure53, NCC Group, and Least Authority. No engagement has been confirmed and no audit has commenced as of September 2026.

Users and organizations deploying GhostWire should understand that: - All security claims in this document reflect self-assessed engineering confidence, not third-party verification. - The cryptographic designs (ML-KEM-768, XChaCha20-Poly1305, X25519 with PFS) are based on

well-studied, published algorithms, but their integration into GhostWire has not been independently audited. - Organizations requiring certified security assurance should plan for an external audit before deploying GhostWire in production environments handling sensitive data.

---

16. Further Reading & Resources

- EFF Surveillance Self-Defense
- OWASP Top 10
- GhostWire Advanced Security
- NIST Post-Quantum Cryptography Standards
- Signal Protocol Design

End of Chapter

Protocol Adapters

Table of Contents

1. What is a Protocol Adapter?
2. Supported & Planned Adapters
3. How Adapters Work (Plain & Technical)
4. The Adapter Registry
5. Real-World Bridging Scenarios
6. Configuration & Code Examples
7. Visual Guide: Adapter Topologies
8. Companion Projects
9. Developer Notes & API Reference
10. Best Practices & Anti-Patterns
11. Troubleshooting & FAQ
12. Further Reading & Resources

---

1. What is a Protocol Adapter?

- **Plain:** A protocol adapter is like a translator that lets GhostWire talk to other messaging networks (like Matrix, Meshtastic, Briar).
- **Technical:** Adapters are software modules that translate messages and events between GhostWire and other protocols, enabling cross-network messaging, group chat, and file sharing.

---

2. Supported & Planned Adapters

Adapter	Status	Notes / Features
Matrix	Implemented	Enabled with the `matrix-bridge` feature flag; basic form today
Meshtastic	Implemented	Erasure coding: 3 data + 2 parity shards, any 3 of 5 reconstruct, fits the 242-byte LoRa frame limit
Briar	Planned	Contact-based messaging, offline queuing, groups
Reticulum bridge	Planned/experimental	Feature `reticulum`; experimental bridge to Reticulum mesh
Tor	Not implemented	No Tor bridge exists yet
Custom	Supported	Build your own adapter via the adapter registry

---

3. How Adapters Work (Plain & Technical)

Plain-Language

- Adapters "translate" messages so GhostWire can talk to other networks.
- You can bridge a GhostWire mesh to Matrix, Meshtastic, or Briar, sharing messages and files.

Technical

- Each adapter implements the `ProtocolAdapter` trait in Rust: `rust pub trait ProtocolAdapter { fn send(&self, msg: Message) -> Result<(), AdapterError>; fn receive(&self) -> Option<Message>; fn connect(&self) -> Result<(), AdapterError>; // ... }`
- Adapters handle translation, deduplication, relay, and group management.
- **Note on Briar:** Internal structs inspired by Briar exist in the codebase, but a real Briar bridge has not been built yet. The "Briar" adapter remains in the Planned state.

---

4. The Adapter Registry

GhostWire uses an adapter registry to manage ProtocolAdapter instances at runtime. To add a new bridge adapter:

1. Implement the `ProtocolAdapter` trait for your adapter.
2. Register the adapter at startup via the registry.
3. The backend will load and initialize the adapter automatically.
4. Test both directions: GhostWire-to-external and external-to-GhostWire.

The registry is the single entry point for all adapters. It ensures that adapters are initialized in the correct order, that dependencies are resolved, and that failed adapters do not crash the backend. See the Matrix and Meshtastic adapters as reference implementations.

---

5. Real-World Bridging Scenarios

Disaster Response

- **Scenario:** GhostWire mesh bridges to Meshtastic LoRa radios for long-range communication.
- **Outcome:** First responders can relay messages between phone users and LoRa devices.
- **Quotable:** "GhostWire to Meshtastic lets a phone talk over LoRa through a nearby relay."

Activist Network

- **Scenario:** Protesters use GhostWire to bridge to Matrix for global reach.
- **Outcome:** Local mesh messages are relayed to Matrix rooms, connecting to the outside world.

Rural Community

- **Scenario:** GhostWire connects to Briar for secure, contact-based messaging.
- **Outcome:** Villagers can chat securely, even offline. (Note: Briar bridge is Planned, not yet implemented.)

---

6. Configuration & Code Examples

Enabling/Disabling Adapters (Config)

```
[adapters]
briar = false
meshtastic = true
matrix = true  # Requires the matrix-bridge feature flag
custom = true
```

Using Adapters in Code

```
let matrix = MatrixAdapter::new();
let meshtastic = MeshtasticAdapter::new();
backend.add_adapter(Box::new(matrix));
backend.add_adapter(Box::new(meshtastic));
```

Web UI Example

- Go to Settings > Adapters
- Toggle available adapters on/off
- See real-time status and diagnostics

---

7. Visual Guide: Adapter Topologies

```
graph TD;
  GhostWire["GhostWire Mesh"] -- Matrix Adapter --> Matrix["Matrix Network"]
```

```
GhostWire -- Meshtastic Adapter --> Meshtastic["Meshtastic Network"]
GhostWire -- Briar Adapter --> Briar["Briar Network (planned)"]
```

---

8. Companion Projects

Two companion projects extend the adapter ecosystem:

- **ghostwire-dtn:** A delay-tolerant networking crate that provides store-and-forward capabilities for adapter-based bridges. Useful when the target network does not support real-time messaging.
- **sphinx-rs:** A Rust implementation of the SPHINX onion-routing protocol, used for metadata protection across adapter bridges.

---

9. Developer Notes & API Reference

- **Implementing a new adapter:**
  - Implement the `ProtocolAdapter` trait.
  - Handle message translation, deduplication, and relay.
  - Register the adapter in the backend via the adapter registry.
  - Test both directions (GhostWire-to-external and external-to-GhostWire).
- **Reference implementations:** See the Matrix adapter (`matrix-bridge` feature) and Meshtastic adapter (erasure coding) for working examples.
- **API Reference:**
  - See Developer Guide for full trait and API docs.

---

10. Best Practices & Anti-Patterns

Best Practices

- Test adapters in isolated environments before production.
- Keep adapters updated for protocol changes.
- Monitor adapter health and logs.
- Test both directions of message flow.
- Use the adapter registry for all adapter registration.

Anti-Patterns

- Relying on a single adapter for all bridging.
- Disabling security features on adapters.
- Ignoring protocol updates.
- Registering adapters outside the registry.

---

11. Troubleshooting & FAQ

Problem	Solution
Can't connect to Matrix	Check credentials, server URL, and network; verify `matrix-bridge` feature flag is enabled
Briar messages not relayed	Briar bridge is Planned; not yet implemented
Meshtastic bridge fails	Check LoRa hardware and adapter config; verify erasure coding parameters
Duplicate messages	Ensure deduplication is enabled
Adapter not loading	Check that the adapter is registered via the adapter registry

---

12. Further Reading & Resources

- Matrix Protocol
- Briar Project
- Meshtastic
- GhostWire Developer Guide
- ghostwire-dtn
- sphinx-rs

End of Chapter

Frequently Asked Questions (FAQ)

Table of Contents

1. General Questions
2. Getting Started
3. Technical & Developer Questions
4. Security & Privacy
5. Real-World Scenarios
6. Troubleshooting & Support
7. Visuals: Common Flows
8. Further Reading & Resources

---

1. General Questions

Q: What is GhostWire? A: A modular, privacy-focused mesh networking and messaging platform supporting multiple transports and advanced security.

Q: Who can use GhostWire? A: Anyone. It is designed for activists, disaster responders, rural communities, privacy enthusiasts, and developers.

Q: Is GhostWire free and open-source? A: Yes. The code is on GitHub.

Q: What platforms are supported? A: Linux, Windows, and macOS. Mobile platform support is under active development.

Q: Who is GhostWire for? A: GhostWire serves six main groups: disaster responders who need to coordinate when cell towers fail, privacy advocates who want no logs and no traces, rural communities where 2.6 billion people lack reliable internet, activists and journalists facing internet shutdowns, whistleblowers who need ephemeral sessions and stealth communication, and developers and researchers who want an AGPLv3 Rust codebase to study and build on.

---

2. Getting Started

Q: How do I install and run GhostWire? A: See the Getting Started chapter. You will need Rust, Node.js, and npm for the full stack, or you can try the web demo if available.

Q: Do I need to be a developer? A: No. There are easy-to-use web and mobile interfaces planned for future releases.

Q: Can I join a mesh without the internet? A: Yes. Use Bluetooth, WiFi, or LoRa to connect locally.

---

3. Technical & Developer Questions

Q: How do I add a new transport or adapter? A: See the Developer Guide. Implement the relevant trait (`Transport` or `ProtocolAdapter`) and register it in the backend.

Q: Is there a plugin system? A: Yes. GhostWire is designed for modularity and extension.

Q: How do I contribute code? A: Fork the repo, make your changes, and submit a pull request. See the Contributing chapter for details.

Q: Which transports work today? A: Five transports are functional: WiFi Direct, LoRa, stealth TCP, WebRTC, and Reticulum. Bluetooth LE support is in progress. The system also includes Traffic Armor, which camouflages traffic and auto-switches transports when one is blocked, using Thompson Sampling to pick the most energy-efficient route.

Q: How does the device hierarchy work? A: GhostWire organizes devices into tiers by available RAM. A phone with 6GB or less handles 20 peers and 1GB of storage. A tablet with 8GB or less handles 30 peers. A Raspberry Pi with 4GB or less handles 15 peers. A laptop with 16GB or less handles 50 peers and 5GB of storage. A server with more than 16GB handles 100 or more peers and 50GB or more of storage. Lower-power devices can offload relay and storage duties to a nearby laptop or server, keeping the mesh stable as it grows.

Q: What AI runs on devices and does it send my data somewhere? A: GhostWire runs AI models directly on your device. A lightweight LightGBM classifier examines each message in about 77 microseconds on a Raspberry Pi to pick the best transport and route. A Gemma 4 analytical model runs every 30 seconds to update the network map. A graph neural network handles routing decisions. Your data does not leave your device. Model files download to your device via

a distributed hash table, and the hashes are verified against known-good references. This is not federated learning, because no gradients or raw data are sent to a central server.

Q: Which protocol bridges work today versus planned? A: The Matrix bridge is implemented behind a feature flag and provides basic connectivity. The Meshtastic bridge is implemented with erasure coding, using 3 data and 2 parity shards so any 3 of 5 can reconstruct a message, which works around the 242-byte LoRa frame limit. The Briar bridge is planned and not yet available. The Reticulum bridge is planned and experimental behind a feature flag. Tor integration is not implemented.

Q: Has GhostWire been audited? A: An external security audit is planned but has not yet been conducted. Candidate auditors include Cure53, NCC Group, and Least Authority. No audit results are available at this time.

Q: Is my battery drained by relaying? A: GhostWire is designed to be battery-aware. When your battery is charging, relaying runs without restrictions. Above 50%, normal operation applies. Between 20% and 50%, cover traffic is reduced by 30%. At 20% or below, the device drops to the most minimal privacy tier and limits relaying. The system also uses energy-aware transport selection, preferring transports that drain less battery.

---

4. Security & Privacy

Q: How secure is GhostWire? A: All messages are end-to-end encrypted. The system uses post-quantum ML-KEM-768 key encapsulation, XChaCha20-Poly1305 encryption, and perfect forward secrecy. See the Security chapter for details.

Q: What if my device is seized? A: Keys are encrypted on disk. Use device encryption for extra safety. A duress PIN can protect against coercion, and a dead man's switch can wipe keys if you fail to check in.

Q: Can I disable security features? A: Not recommended. Only for advanced users in test environments.

Q: What is a privacy tier? A: GhostWire assigns privacy tiers based on device type and battery level. Servers operate at L3, laptops and tablets at L2, and phones and Raspberry Pis at L1. When battery drops, the device drops one tier, reducing its visibility and relaying activity.

---

5. Real-World Scenarios

Q: How does GhostWire help in a disaster? A: It enables communication when infrastructure is down, using LoRa, WiFi, and Bluetooth.

Q: Can GhostWire work without internet connectivity? A: Yes. It uses Bluetooth, WiFi, and LoRa to maintain communication during infrastructure failures.

Q: Has GhostWire been used in real-world events? A: See the Case Studies chapter for detailed stories.

---

6. Troubleshooting & Support

Problem	Solution
Can't find other nodes	Check transport settings, try another method
Messages not sending	Ensure at least one transport is active
Web UI won't load	Check backend is running, try npm install
LoRa not working	Check hardware, drivers, and permissions
Security warning	Ensure you are using the latest version

- **Logs:** Check backend logs for errors.
- **Community:** Ask for help on GitHub or project chat.

---

7. Visuals: Common Flows

```
graph TD;
  User["User"] --> UI["Web UI/CLI"]
  UI --> Backend["Backend"]
  Backend --> Mesh["Mesh Network"]
  Mesh --> Internet["Internet (optional)"]
```

---

8. Further Reading & Resources

- GhostWire GitHub
- Mesh Networking 101
- Security & Privacy
- Developer Guide
- Device Hierarchy & Power-Aware Scaling
- AI Routing & On-Device Intelligence

End of Chapter

Contributing & Community

Table of Contents

1. Welcome & Philosophy
2. How Anyone Can Contribute
3. Code Contributions (Technical)
4. Writing, Design, and Outreach
5. Community Guidelines & Code of Conduct
6. Example Contribution Journey
7. Visuals: Contribution Flow

8. Best Practices for Contributors
9. Further Reading & Resources

---

1. Welcome & Philosophy

GhostWire is open to everyone, whether you are a coder, writer, designer, tester, or just curious. Here is how you can help build the future of secure, decentralized communication.

---

2. How Anyone Can Contribute

- **Test the app:** Try GhostWire and give feedback.
- **Report bugs:** Found a problem? Open an issue on GitHub.
- **Write docs:** Help make guides clearer for everyone.
- **Design:** Improve the UI/UX or create graphics.
- **Spread the word:** Share GhostWire with your community.
- **Join discussions:** Help shape the roadmap and features.

---

3. Code Contributions (Technical)

- **Code style:**
  - Rust: Follow rustfmt and clippy guidelines.
  - JS/TS: Use Prettier and ESLint.
- **How to contribute:**
  1. Fork the repo on GitHub.
  2. Create a feature branch.
  3. Make your changes and add tests.
  4. Run all tests and linters.
  5. Submit a pull request with a clear description.
- **Review process:**
  - All PRs are reviewed for security, style, and clarity.
  - Feedback is constructive and focused on improvement.

---

4. Writing, Design, and Outreach

- **Docs:** Improve guides, add visuals, translate content.
- **Design:** Create logos, UI mockups, infographics.
- **Outreach:** Write blog posts, give talks, organize events.

---

5. Community Guidelines & Code of Conduct

- **Be respectful:** Treat everyone with kindness and respect.
- **Be inclusive:** Welcome contributors of all backgrounds and skill levels.

- **No harassment:** Zero tolerance for abuse or discrimination.
- **Help others:** Support newcomers and share knowledge.

---

6. Example Contribution Journey

The following are hypothetical examples showing common paths new contributors take. They are not real stories but illustrate what a typical contribution looks like.

Example 1: Documentation fix A contributor notices a typo in the FAQ chapter. They fork the repo, edit the markdown file, and submit a pull request. The fix is reviewed and merged within a day. This is a common first contribution because it requires minimal code knowledge.

Example 2: Transport implementation A developer wants to add a new transport. They implement the `Transport` trait, write unit tests, and register it in the adapter registry. After code review and a few rounds of feedback, the transport is merged. This path requires deeper Rust knowledge.

Example 3: Design contribution A designer creates a new icon set for the web UI. They submit the assets through a pull request with a preview. The design team reviews the assets for consistency with the project style guide.

---

7. Visuals: Contribution Flow

```
graph TD;
  Contributor["You"] --> Fork["Fork Repo"]
  Fork --> Branch["Create Branch"]
  Branch --> PR["Submit Pull Request"]
  PR --> Review["Code Review"]
  Review --> Merge["Merged!"]
```

---

8. Best Practices for Contributors

- Communicate clearly and kindly.
- Ask questions if you are unsure.
- Test your changes before submitting.
- Review the documentation and guidelines.
- Celebrate your contributions.

---

9. Further Reading & Resources

- GhostWire GitHub
- CONTRIBUTING.md
- Code of Conduct
- Open Source Guides

End of Chapter

Deployment & Operations

Table of Contents

1. Overview
2. Deployment Scenarios
3. Assigning Roles by Power and Hardware
4. Step-by-Step Deployment Guides
5. Docker, Cloud, and Bare-Metal Installations
6. Monitoring, Logging, and Maintenance
7. Visuals: Deployment Topologies
8. Troubleshooting & Recovery
9. Best Practices & Anti-Patterns
10. Further Reading & Resources

---

1. Overview

This chapter covers how to deploy, operate, and maintain GhostWire in a variety of real-world scenarios, from a single home node to a city-wide mesh. Both non-technical and technical readers will find step-by-step guides, visuals, and best practices.

---

2. Deployment Scenarios

Home/Personal Mesh

- **Goal:** Connect a few devices (phones, laptops) for secure messaging at home or in a small group.
- **Steps:**
  1. Download and install the GhostWire app or desktop client.
  2. Start the app and select your preferred transport (WiFi Direct, LoRa).
  3. Invite nearby devices to join your mesh.

Community/Neighborhood Mesh

- **Goal:** Connect dozens of devices across a neighborhood or event.
- **Steps:**
  1. Deploy GhostWire on laptops, phones, and LoRa relays.
  2. Use WiFi Direct and LoRa for coverage.
  3. Assign some nodes as relays for better reach.

Disaster/Field Deployment

- **Goal:** Restore communication after infrastructure failure.
- **Steps:**
  1. Deploy LoRa nodes at key locations (battery/solar powered).
  2. Use store-and-forward for offline delivery.
  3. Federate with other meshes if possible.

Enterprise/Federated Mesh

- **Goal:** Connect multiple sites, enable advanced security, and integrate with existing systems.
- **Steps:**
  1. Deploy GhostWire on servers (bare-metal or cloud).
  2. Use Docker or Kubernetes for scaling.
  3. Integrate with protocol adapters for interoperability.

---

3. Assigning Roles by Power and Hardware

GhostWire assigns roles based on device capability. The right device in the right role makes the mesh faster, more resilient, and easier to operate.

Device Classes

Class	RAM	Max Peers	Typical Device
Phone	<=6GB	20	Consumer smartphone
Tablet	<=8GB	30	iPad, Android tablet
Raspberry Pi	<=4GB	15	Pi 4, Pi 5
Laptop	<=16GB	50	Consumer laptop, ultrabook
Server	>16GB	100+	Rack server, cloud VM

Roles

- **Full Node:** Stores the complete mesh state. Participates in routing and relay. Requires the most resources.
- **Relay:** Forwards messages between nodes. Relays are the backbone of the mesh.
- **Leaf:** Connects to the mesh but does not relay for others. Lowest resource requirement.
- **Sensor:** Publishes telemetry or data feeds. Lightweight, often battery-powered.
- **Replicator:** Maintains copies of critical data for disaster recovery. Operates on servers or dedicated hardware.

Role Assignment Principles

- "Assign key nodes (with good power and connectivity) as relays."
- "Use devices with more RAM/CPU for relays."
- Relays carry no trust authority. A relay forwards packets but cannot read their contents.

Battery States and Role Behavior

GhostWire adjusts node behavior based on battery level:

Battery State	Behavior
Charging	Unrestricted. Full role, full cover traffic.
Above 50%	Normal operation. All roles available.
20-50%	Cut cover traffic by 30% to conserve energy.
20% or below	Drop to privacy L0/L1. Minimize all non-essential activity.

Solar and Battery-Powered LoRa Relays

LoRa relays in remote deployments run on solar panels with battery backup. These nodes prioritize store-and-forward messaging and minimize radio wake time. During low-light conditions, the relay drops to L1 privacy and reduces transmission frequency.

Supernode Election Operations

- Elect nodes with high uptime, stable IP addresses, and good bandwidth as "Relay Hubs."
- Relay Hubs form the backbone of the mesh and handle inter-region traffic.
- Regional sharding is planned for v3/v4, where each geographic region operates its own DHT shard connected to a global backbone.
- Supernode election uses uptime history, bandwidth measurements, and peer reputation scores.

ESP32 Gateway-Bridged Deployments

- **$20 ESP32 mini-mesh:** A low-cost ESP32 board running a lightweight mesh client bridged to WiFi. Ideal for adding legacy sensors or very low-power nodes to a WiFi-based mesh.
- **$100 Raspberry Pi gateway:** A Pi running GhostWire with a WiFi adapter and LoRa module. Bridges between WiFi Direct, LoRa, and the broader internet.
- **Server-class backbone:** A dedicated server acting as the root relay and federation point. Handles the highest peer counts and provides the bridge to external networks.

OEM Battery Optimization Note

Many consumer devices sold in Kenya and East Africa come from OEMs such as Samsung, Xiaomi, Huawei, OPPO, and Transsion. These manufacturers aggressively optimize battery usage and may kill background processes. Follow the dontkillmyapp guidance to ensure GhostWire continues running in the background. Whitelist the GhostWire process in battery optimization settings and disable aggressive app standby modes.

---

4. Step-by-Step Deployment Guides

Local (Laptop/Desktop)

1. Download and install GhostWire.
2. Run the backend and web UI.
3. Join or create a mesh.

Docker

1. Pull the GhostWire Docker image:

   ```
   docker pull phantomojo/ghostwire:latest
   ```

2. Run the container:

   ```
   docker run -d -p 3000:3000 -p 9000:9000 phantomojo/ghostwire:latest
   ```

3. Access the web UI at http://localhost:3000

Cloud (AWS, GCP, Azure)

1. Provision a VM or container instance.
2. Install Docker or run natively.
3. Open required ports (3000, 9000, LoRa if needed).
4. Secure with firewalls and access controls.

Bare-Metal

1. Install Rust and Node.js.
2. Build and run GhostWire as per Getting Started.
3. Set up systemd service for auto-restart.

---

5. Docker, Cloud, and Bare-Metal Installations

- **Docker:** Easiest for quick deployment and scaling.
- **Cloud:** Use for global reach, federation, and integration.
- **Bare-Metal:** Best for custom hardware, edge, or offline use.

---

6. Monitoring, Logging, and Maintenance

- **Monitoring:**
  - Use built-in web UI dashboard for node status and health.
  - Integrate with Prometheus/Grafana for advanced metrics.
- **Logging:**
  - Backend logs to file and stdout.
  - Use log rotation for long-term deployments.
- **Maintenance:**
  - Regularly update software.

- Backup configuration and keys.
- Test failover and recovery procedures.

---

7. Visuals: Deployment Topologies

```
graph TD;
  Home["Home Node"] -- WiFi Direct --> Laptop["Laptop"]
  Laptop -- LoRa --> Relay["LoRa Relay"]
  Relay -- LoRa --> Village["Remote Village Node"]
  Laptop -- WebRTC --> Browser["Browser"]
  Laptop -- stealth TCP --> Internet["Internet Node"]
  Cloud["Cloud Server"] -- Federation --> OtherMesh["Other Mesh"]
  ESP32["ESP32 Gateway"] -- WiFi --> Laptop
  Pi["Raspberry Pi"] -- LoRa --> Relay
```

---

8. Troubleshooting & Recovery

Problem	Solution
Node won't start	Check logs, ensure dependencies are installed
Can't connect to mesh	Check transport settings, firewall, and ports
Docker container fails	Check image version, logs, and port mapping
Cloud instance unreachable	Check security groups, firewall, and DNS
LoRa not working	Check hardware, drivers, and permissions
OEM battery killing process	Follow dontkillmyapp guidance, whitelist app

---

9. Best Practices & Anti-Patterns

Best Practices

- Use Docker for easy scaling and updates.
- Monitor node health and logs.
- Regularly backup configuration and keys.
- Test failover and recovery.
- Secure cloud deployments with firewalls and access controls.
- Assign relays based on device capability, not just availability.
- Follow dontkillmyapp guidance on OEM-optimized Android devices. ### Anti-Patterns
- Ignoring updates or logs.
- Using default passwords or open ports.
- Not testing disaster recovery.
- Placing low-power devices as relays.

---

10. Further Reading & Resources

- Docker Documentation
- Prometheus
- Grafana
- GhostWire Developer Guide
- dontkillmyapp

End of Chapter

Advanced Security & Threat Response

Table of Contents

1. Overview
2. Threat Modeling (Plain & Technical)
3. Incident Response & Forensics
4. Security Module Configuration
5. Audit & Compliance
6. Example Attack/Defense Scenarios
7. Visuals: Threat Flows & Response
8. Actionable Checklists
9. Further Reading & Resources

---

1. Overview

This chapter is for those who want to go beyond the basics, security engineers, admins, and anyone responsible for defending a GhostWire mesh. It covers advanced threat modeling, incident response, forensics, and real-world attack/defense scenarios, with both plain-language and technical explanations.

---

2. Threat Modeling (Plain & Technical)

- **Plain:** Threat modeling is thinking ahead about what could go wrong and how to stop it.
- **Technical:** Use frameworks like STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to analyze risks.
- **Visual:**

```
graph TD;
  Attacker -->|Spoofing| Node
  Attacker -->|Tampering| Message
```

```
Attacker -->|DoS| Mesh
Attacker -->|Traffic Analysis| Network
```

---

3. Incident Response & Forensics

- **Preparation:**
  - Document your mesh topology and key nodes.
  - Regularly backup configuration and keys.
  - Configure dead man's switch timeouts for all relay nodes.
  - Distribute Shamir secret sharing shares to trusted contacts.
- **Detection:**
  - Monitor logs and alerts for suspicious activity.
  - Use anomaly detection tools if available.
  - Watch for unexpected peer count spikes (Sybil indicator).
  - Monitor for cover traffic pattern deviations.
- **Response:**
  - Isolate compromised nodes.
  - Rotate keys and update blacklists.
  - Communicate securely with trusted nodes.
  - Activate duress PIN mode if coercion is suspected.
  - Wipe device if seizure is confirmed and dead man's switch has not triggered.
- **Forensics:**
  - Collect logs and evidence.
  - Analyze attack vectors and entry points.
  - Report findings to the community.
  - Preserve encrypted volumes for post-incident analysis.

Duress PIN Incident Response

If a device running GhostWire is seized or the operator is coerced: 1. **Enter duress PIN:** This activates plausible-deniability mode. The device presents a harmless or empty view of the mesh. Real identity and keys remain cryptographically hidden. 2. **Observe behavior:** Under duress mode, the device appears to have no active mesh connections and no stored messages. An adversary inspecting the device cannot distinguish this from a device that was never configured. 3. **Dead man's switch:** If the device goes silent beyond the configured timeout after seizure, the dead man's switch automatically triggers. It wipes local keys and notifies trusted contacts. 4. **Post-seizure recovery:** Once the operator is safe, they can use Shamir secret sharing shares to reconstruct their identity on a new device.

---

4. Security Module Configuration

- **SybilDefense:** Adjust proof-of-work/stake parameters.
- **QuotaEnforcer:** Set rate limits for messages and actions.
- **BlacklistManager:** Add/remove nodes as needed.
- **TrafficObfuscator:** Enable/disable padding and timing obfuscation.
- **KeyManager:** Schedule key rotation and backups.

---

5. Audit & Compliance

- **Audit:**
  - Regularly review logs and configuration.
  - Use automated tools for compliance checks.
- **Compliance:**
  - Follow local laws and regulations for encryption and radio use.
  - Document security policies and procedures.

External Audit Status

An external security audit has NOT been performed. GhostWire's current protections are self-reviewed engineering controls. The development team has conducted internal reviews of the cryptographic implementations, module interactions, and threat model coverage.

External audit is PLANNED. Candidate auditing firms include Cure53, NCC Group, and Least Authority. No engagement has been confirmed and no audit has commenced as of September 2026.

Organizations requiring certified security assurance should plan for an external audit before deploying GhostWire in production environments handling sensitive data.

---

6. Example Attack/Defense Scenarios

- **Sybil Attack:**
  - **Attack:** Adversary floods mesh with fake nodes.
  - **Defense:** Enable SybilDefense, require proof-of-work/stake, monitor for anomalies.
- **DoS Attack:**
  - **Attack:** Flood of messages to overwhelm nodes.
  - **Defense:** Set quotas, enable rate limiting, blacklist offenders.
- **Traffic Analysis:**
  - **Attack:** Adversary infers communication patterns by observing timing, volume, and packet sizes.
  - **Defense:** Enable traffic obfuscation, use stealth transports. Traffic Armor disguises WebRTC traffic to look like a video call. Stealth TCP automatically escalates to alternative protocols when blocked. SPHINX onion routing ensures no single node knows both sender and recipient. Cover traffic generates dummy packets to mask real communication patterns.
- **Key Compromise:**
  - **Attack:** Device is seized, keys are stolen.
  - **Defense:** Use encrypted storage, rotate keys, wipe device if needed. Enable duress PIN mode for plausible deniability. Use dead man's switch for automatic key wiping on extended silence.
- **Device Seizure with Coercion:**
  - **Attack:** Adversary physically seizes a device and forces the operator to reveal credentials.
  - **Defense:** Duress PIN activates plausible-deniability mode. Real keys remain encrypted and inaccessible. Dead man's switch triggers if the device goes silent. Shamir secret

sharing allows recovery on a new device without the seized one.

---

7. Visuals: Threat Flows & Response

```
graph TD;
  Attacker["Attacker"] -->|Sybil| Mesh["Mesh Network"]
  Attacker -->|DoS| Node["Node"]
  Attacker -->|Traffic Analysis| Network["Network"]
  Defender["Defender"] -->|Key Rotation| Node
  Defender -->|Blacklist| Mesh
  Defender -->|Duress PIN| Device["Device"]
  Defender -->|Cover Traffic| Obfuscation["Traffic Obfuscation"]
```

---

8. Actionable Checklists

- ☐ Document mesh topology and key nodes
- ☐ Regularly backup configuration and keys
- ☐ Monitor logs and alerts
- ☐ Enable all security modules
- ☐ Review and update blacklists
- ☐ Test incident response procedures
- ☐ Test duress PIN mode before deployment
- ☐ Configure dead man's switch timeout
- ☐ Verify cover traffic is active on relay nodes
- ☐ Confirm stealth transports are configured
- ☐ Distribute Shamir secret sharing shares to trusted contacts
- ☐ Review external audit status and plan accordingly

---

9. Further Reading & Resources

- OWASP Incident Response
- NIST Cybersecurity Framework
- GhostWire Security & Privacy
- Signal Protocol Design
- SPHINX Onion Routing

End of Chapter

Performance & Scaling

Table of Contents

1. Overview
2. Optimization Tips
3. Benchmarks & Metrics
4. Tiered Scaling Architecture
5. Energy-Aware Transport Selection
6. AI Latency Context
7. Troubleshooting Performance
8. Visuals: Performance Flows
9. Best Practices & Anti-Patterns
10. Further Reading & Resources

---

1. Overview

This chapter covers how to optimize GhostWire for speed, reliability, and large-scale deployments. Both non-technical and technical readers will find practical tips, benchmarks, and troubleshooting guides, with visuals.

---

2. Optimization Tips

- **Use multiple transports:** Combine WiFi Direct, LoRa, and Bluetooth for best coverage.
- **Prioritize nodes:** Assign key nodes (with good power and connectivity) as relays.
- **Tune quotas:** Adjust rate limits for your mesh size and expected traffic.
- **Monitor health:** Use the web UI or Prometheus/Grafana for real-time stats.
- **Upgrade hardware:** Use devices with more RAM/CPU for relays.

---

3. Benchmarks & Metrics

Scenario	Devices	Avg. Latency	Max Throughput
Home Mesh	5	50ms	1 Mbps
Community Mesh	50	100ms	5 Mbps
Disaster Field	20	200ms	500 Kbps
Enterprise Mesh	200+	150ms	10 Mbps

- **Metrics to monitor:**
  - Latency (ms)
  - Throughput (Mbps)
  - Node uptime (%)
  - Message delivery rate (%)
  - Relay hop count
  - Cover traffic overhead (bytes/sec)
  - Battery level of mobile nodes
  - Peer count per node

– Transport switching frequency

---

4. Tiered Scaling Architecture

A flat mesh becomes unstable beyond a certain density. We should introduce a tiered architecture.

The Problem with Flat Meshes

In a flat mesh, every node participates in routing. As node count grows, routing tables expand, message latency increases, and the network becomes fragile. A community mesh of 50 nodes works well. A city-scale mesh of 500+ nodes requires structure.

Supernode and Relay Hub Election

- Elect nodes with high uptime, stable IP addresses, and good bandwidth as "Relay Hubs."
- Relay Hubs form the backbone of the mesh and handle inter-region traffic.
- Supernode election uses uptime history, bandwidth measurements, and peer reputation scores.
- Relay Hubs are not trust authorities. They forward packets but cannot read their contents.

Regional DHT Sharding (v3/v4 Plan)

- Each geographic region operates its own DHT shard.
- Regional shards connect to a global backbone through designated gateway nodes.
- This reduces per-node routing overhead and improves lookup latency.
- The hierarchy is: Leaf nodes -> Regional relays -> Supernode Relay Hubs -> Global backbone.

Scaling Targets

Tier	Node Count	Architecture
Small	5-20	Flat mesh
Medium	20-100	Relay-based with elected hubs
Large	100-1000	Regional sharding with supernodes
Enterprise	1000+	Hierarchical supernodes with global backbone

---

5. Energy-Aware Transport Selection

GhostWire selects transports based on energy efficiency using Thompson Sampling, a multi-armed bandit algorithm that balances exploration and exploitation.

Energy-Efficiency Weights

Each transport has an energy-efficiency weight that influences selection when battery is constrained:

Transport	Energy Weight	Notes
BLE (Bluetooth Low Energy)	0.30	Very low power, short range
LoRa	0.35	Low power, long range, low bandwidth
Reticulum	0.25	Adaptive, moderate power
WiFi Direct	0.05	Higher power, high bandwidth
TCP/QUIC	0	Internet-bound, highest power

How Thompson Sampling Works

1. Each transport starts with a prior distribution over its expected energy cost.
2. After each transmission, the observed energy cost updates the distribution.
3. The algorithm samples from each transport's posterior and selects the one with the highest sampled efficiency.
4. Over time, the algorithm converges on the most energy-efficient transport for the current conditions.

Battery-Aware Behavior

- **Charging:** Unrestricted. All transports available. Full cover traffic.
- **Above 50%:** Normal operation. Thompson Sampling selects transports normally.
- **20-50%:** Cut cover traffic by 30%. Bias Thompson Sampling toward BLE and LoRa.
- **20% or below:** Drop to privacy L0/L1. Minimize all non-essential activity. Prefer LoRa only.

---

6. AI Latency Context

GhostWire integrates lightweight AI for message routing, anomaly detection, and predictive mesh optimization. Understanding AI latency helps plan deployment expectations.

Inference Timings

- **LightGBM inference:** 77 microseconds per message on a Raspberry Pi. Used for routing decisions and anomaly classification.
- **ML-KEM-768 handshake:** 890 microseconds on-device. Post-quantum key establishment.
- **Gemma 4 analytical pass:** Every 30 seconds. Used for mesh health analysis and traffic pattern recognition.

AI Architecture

- **LightGBM:** Gradient-boosted decision trees for fast classification. Runs locally on every node.
- **Gemma 4:** Lightweight analytical model for periodic mesh assessment. Runs on server-class nodes.
- **GNN (Graph Neural Network):** Planned for future routing optimization across the mesh topology.
- **PRoPHET:** Probabilistic routing protocol for delay-tolerant networks. Used in disaster/field scenarios.

Telemetry

- Telemetry data is sent up (opt-in) to improve models.
- Model files are distributed down via DHT hash and P2P sync.
- This is not federated learning. Models are shared as files, not as gradient updates.

---

7. Troubleshooting Performance

Problem	Solution
High latency	Check transport health, upgrade relays
Dropped messages	Increase quotas, check logs
Node offline	Check power/network, use redundant relays
Slow UI	Upgrade device, close unused apps
Battery draining fast	Check transport selection, reduce cover traffic
Relay overload	Elect new relay hubs, rebalance peers
Transport switching too often	Adjust Thompson Sampling exploration rate

---

8. Visuals: Performance Flows

```
graph TD;
  User["User"] --> Node["Node"]
  Node --> Relay["Relay Node"]
  Relay --> Mesh["Mesh Network"]
  Mesh --> Internet["Internet (optional)"]
  Node --> LB["LightGBM (77us)"]
  Node --> KEM["ML-KEM-768 (890us)"]
  Relay --> Super["Supernode Relay Hub"]
  Super --> Region["Regional DHT Shard"]
  Region --> Global["Global Backbone"]
```

---

9. Best Practices & Anti-Patterns

Best Practices

- Monitor mesh health and performance.
- Use redundant relays for reliability.
- Regularly update software and hardware.
- Segment large meshes for manageability.
- Use tiered architecture beyond 100 nodes.
- Enable energy-aware transport selection on battery-powered nodes.
- Monitor AI inference latency on resource-constrained devices. ### Anti-Patterns
- Ignoring performance metrics.
- Using outdated hardware for relays.

- Not testing at scale before deployment.
- Running flat meshes beyond their density limit.
- Ignoring battery state when assigning roles.

---

10. Further Reading & Resources

- Prometheus
- Grafana
- GhostWire Developer Guide
- Thompson Sampling
- LightGBM

End of Chapter

Developer Guide

Table of Contents

1. Overview
2. Extending GhostWire (Transports, Adapters, Modules)
3. Adding a Bridge Adapter
4. Extending AI Routing
5. API Reference & Hooks
6. Plugin System & Architecture
7. Testing & CI/CD
8. Visuals: Developer Flows
9. Best Practices & Anti-Patterns
10. Further Reading & Resources

---

1. Overview

This chapter is for developers who want to extend GhostWire, add new transports, adapters, modules, or contribute to the core. Both non-technical and technical readers will find step-by-step guides, API references, and best practices, with visuals.

---

2. Extending GhostWire (Transports, Adapters, Modules)

Adding a New Transport

1. Implement the `Transport` trait in Rust.
2. Register your transport in the backend.
3. Add configuration options to the web UI/CLI.

4. Test with simulated and real devices.

Adding a Protocol Adapter

1. Implement the `ProtocolAdapter` trait.
2. Handle message translation, deduplication, and relay.
3. Register the adapter in the backend.

Adding a Security Module

1. Implement the relevant trait (e.g., `QuotaEnforcer`).
2. Register and configure in the backend.

---

3. Adding a Bridge Adapter

To add a new bridge adapter that connects GhostWire to an external protocol:

1. **Implement the `ProtocolAdapter` trait.** Define how your adapter sends, receives, and connects to the external network. See the Matrix adapter (`matrix-bridge` feature) and Meshtastic adapter (erasure coding) for reference implementations.
2. **Register the adapter via the adapter registry.** The registry is the single entry point for all adapters. Registering at startup ensures the adapter is initialized correctly and that dependencies are resolved before the backend starts serving traffic.
3. **Test both directions.** Verify that messages flow from GhostWire to the external network and from the external network back to GhostWire. Deduplication and translation must work in both directions.
4. **Add configuration options.** Add a `[adapters]` entry in the TOML config and expose toggles in the web UI.
5. **Test in isolation.** Use a simulated external network or a local test instance before deploying to a live mesh.

Reference: Matrix Adapter

The Matrix adapter is implemented and enabled with the `matrix-bridge` feature flag. It provides basic bridging to Matrix rooms. Study its implementation for patterns on how to structure a new adapter.

Reference: Meshtastic Adapter

The Meshtastic adapter is implemented with erasure coding (3 data + 2 parity shards, any 3 of 5 reconstruct). It fits messages into the 242-byte LoRa frame limit. This adapter demonstrates how to handle protocol-specific constraints within the `ProtocolAdapter` trait.

---

4. Extending AI Routing

The AI Routing Layer (see Architecture chapter) can be extended in two ways:

- **Routing classifier features:** The LightGBM per-message classifier accepts features such as `battery\_level`, `transport\_metrics`, `message\_size`, and `destination\_type`. To add a new feature, update the feature extraction module and retrain the classifier.
- **Energy-efficiency weights:** The Thompson Sampling transport selector uses energy-efficiency weights (BLE 0.30, LoRa 0.35, Reticulum 0.25, WiFi 0.05, TCP/QUIC 0). To add a new transport, add its weight to the configuration and update the sampling logic.

For deeper changes (new model architecture, new routing layer), see the AI Routing Layer section in the Architecture chapter.

---

5. API Reference & Hooks

- **REST API:**
  - `/api/nodes`: List nodes
  - `/api/messages`: Send/receive messages
  - `/api/transports`: Manage transports
  - `/api/adapters`: Manage adapters
- **WebSocket API:**
  - Real-time message and event updates
- **Hooks:**
  - Pre-send, post-receive, error handling

---

6. Plugin System & Architecture

- **Plugins:**
  - Add new features without modifying core code.
  - Register plugins via config or UI.
- **Architecture:**
  - Modular, with clear interfaces for each component.
  - See Developer Guide for trait definitions.

---

7. Testing & CI/CD

- **Unit tests:**
  - Write tests for each module and trait.
- **Integration tests:**
  - Test end-to-end flows (see test suite).
- **CI/CD:**
  - Use GitHub Actions for automated builds, tests, and deployments.
  - Linting, security checks, and code coverage included.

---

8. Visuals: Developer Flows

```
graph TD;
  Dev["Developer"] --> Code["Write Code"]
  Code --> Test["Run Tests"]
  Test --> CI["CI/CD Pipeline"]
  CI --> Deploy["Deploy to Mesh"]
```

---

9. Best Practices & Anti-Patterns

Best Practices

- Write clear, well-documented code.
- Test all changes before merging.
- Use modular design for easy extension.
- Follow code style and security guidelines.
- Test bridge adapters in both directions.
- Register all adapters through the adapter registry.

Anti-Patterns

- Skipping tests or code review.
- Hardcoding secrets or credentials.
- Ignoring documentation.
- Registering adapters outside the registry.

---

10. Further Reading & Resources

- GhostWire GitHub
- Rust Book
- GitHub Actions

End of Chapter

Case Studies & Scenario Narratives

Table of Contents

1. Overview
2. Hurricane Response Within 200km
3. Community Blackout
4. Rural Village Connectivity
5. Festival Mesh
6. Lessons Learned

7. Visuals: Scenario Topologies
8. Best Practices & Takeaways
9. Further Reading & Resources

---

1. Overview

This chapter presents illustrative deployment scenarios drawn from project materials. These are scenario narratives, not completed real-world rollouts. Each scenario shows how GhostWire could be deployed in a specific context, using the device hierarchy, transport options, and privacy tiers described in other chapters. The scenarios are designed to help planners understand how the system behaves under different conditions.

For context on the personas who would use these systems, see chapter 17_target_users.md. For the device hierarchy that governs role assignment, see chapter 15_device_hierarchy_and_scaling.md.

---

2. Hurricane Response Within 200km

Scenario: A hurricane has damaged infrastructure within a 200km radius. Cell towers are down, internet is unavailable, and first responders need to coordinate across a wide area.

Hierarchy and deployment: - **Phones as leaf nodes:** First responders carry phones configured as leaf nodes. They send and receive messages but do not relay for others, conserving battery. - **Laptops and Raspberry Pi as relays:** Field teams set up laptops and Raspberry Pi devices as relay nodes at staging areas. These devices forward messages between leaf nodes and extend the mesh reach. - **Battery and solar LoRa relays:** LoRa relay nodes are deployed at key locations using battery and solar power. These relays provide long-range connectivity between staging areas that are out of WiFi Direct range. - **Supernode election:** As the response matures, the team elects the most stable nodes (high uptime, stable power, good bandwidth) as supernode relay hubs. These hubs coordinate regional traffic and form the backbone of the emergency mesh.

Transport mix: LoRa for long-range links between relay stations, WiFi Direct for short-range connections between nearby nodes, stealth TCP for any internet-bound traffic that becomes available.

Privacy tier: Nodes operate at L2 (laptop/tablet tier) or L1 (phone tier) depending on battery. LoRa relays with solar power maintain L2. Phones on battery drop to L1.

```
graph TD;
  Responder1["Phone (Leaf)"] -- WiFi Direct --> Laptop1["Laptop (Relay)"]
  Responder2["Phone (Leaf)"] -- WiFi Direct --> Laptop1
  Laptop1 -- LoRa --> Relay1["LoRa Relay (Solar)"]
  Relay1 -- LoRa --> Relay2["LoRa Relay (Battery)"]
  Relay2 -- LoRa --> Responder3["Phone (Leaf)"]
  Laptop1 -- LoRa --> Supernode["Supernode Relay Hub"]
  Supernode -- LoRa --> Shelter["Shelter (Leaf)"]
```

Lessons from this scenario: - LoRa relays are essential for bridging gaps beyond WiFi Direct range. - Solar-powered relays maintain operation longer than battery-only nodes. - Supernode election should happen early so the mesh has a stable backbone.

---

3. Community Blackout

Scenario: A city experiences a prolonged power outage. Residents need to coordinate, share information, and maintain communication without internet or cellular service.

Hierarchy and deployment: - **Phones as leaf nodes:** Residents use phones as leaf nodes for messaging. - **Laptops and Raspberry Pi as relays:** Community centers and key locations deploy laptops and Raspberry Pi devices as relays. These nodes form the mesh backbone. - **Battery and solar LoRa relays:** Solar-powered LoRa relays are placed on rooftops and at community centers to provide neighborhood-wide coverage. - **Supernode election for the blackout city:** The city-wide mesh elects relay hubs based on uptime and connectivity. Nodes with stable power (e.g., community centers with generators) become supernodes. Regional sharding divides the city into zones, each with its own relay hub, connected to a city-wide backbone.

Transport mix: WiFi Direct for neighborhood-level communication, LoRa for inter-neighborhood links, WebRTC for any node that regains partial internet access.

Privacy tier: Laptops and Raspberry Pi relays operate at L2. Phones operate at L1. Nodes on battery below 50% cut cover traffic by 30%.

```
graph TD;
  Resident1["Phone (Leaf)"] -- WiFi Direct --> Laptop1["Laptop (Relay)"]
  Resident2["Phone (Leaf)"] -- WiFi Direct --> Laptop1
  Laptop1 -- WiFi Direct --> Laptop2["Laptop (Relay)"]
  Laptop2 -- LoRa --> Pi1["Raspberry Pi (Relay)"]
  Pi1 -- LoRa --> SolarRelay["Solar LoRa Relay"]
  SolarRelay -- LoRa --> CommunityCenter["Community Center (Supernode)"]
  CommunityCenter -- WiFi Direct --> Resident3["Phone (Leaf)"]
  CommunityCenter -- WebRTC --> Internet["Partial Internet"]
```

Lessons from this scenario: - Community centers with stable power naturally become supernodes. - Regional sharding prevents a single point of failure from collapsing the entire mesh. - Solar LoRa relays are critical for maintaining connectivity during extended outages.

---

4. Rural Village Connectivity

Scenario: A rural village with no internet access needs to share news, alerts, and educational content among homes, schools, and clinics.

Hierarchy and deployment: - **Phones as leaf nodes:** Villagers use phones to send and receive messages. - **Laptops and Raspberry Pi as relays:** A central location (school or clinic) hosts a laptop or Raspberry Pi as a relay node. This relay connects the village mesh. - **Battery and solar LoRa relays:** LoRa relays connect homes to the central relay. Solar-powered relays are placed at the edges of the village to extend coverage. - **Supernode election:** The central relay node (at the school or clinic) becomes the supernode. It maintains the DHT shard for the village and connects to any external meshes via LoRa or available internet.

Transport mix: LoRa for home-to-relay links, WiFi Direct for local connections within the central hub, Reticulum for any external mesh federation.

Privacy tier: Central relay operates at L3 (server tier) if powered continuously. Home nodes operate at L1. Solar relays maintain L2.

```
graph TD;
  Home1["Home (Phone, Leaf)"] -- LoRa --> School["School (Raspberry Pi, Relay)"]
  Home2["Home (Phone, Leaf)"] -- LoRa --> School
  Home3["Home (Phone, Leaf)"] -- LoRa --> SolarRelay["Solar LoRa Relay"]
  SolarRelay -- LoRa --> School
  School -- WiFi Direct --> Clinic["Clinic (Laptop, Relay)"]
  School -- Reticulum --> ExternalMesh["External Mesh"]
  School --> Supernode["Supernode (School)"]
```

Lessons from this scenario: - A single central relay can serve an entire village if placed strategically. - Solar relays at the village edges extend coverage without running power lines. - The school or clinic is a natural supernode location due to reliable power and community trust.

---

5. Festival Mesh

Scenario: A large festival with overloaded cell networks. Attendees need to form a local mesh for updates, safety alerts, and coordination.

Hierarchy and deployment: - **Phones as leaf nodes:** Attendees use phones as leaf nodes. - **Laptops and Raspberry Pi as relays:** Event organizers deploy laptops and Raspberry Pi devices at key points (entrance, stages, info booths) as relays. - **Battery and solar LoRa relays:** LoRa relays are placed at high points to cover the festival grounds. Battery-powered relays are used where solar is not practical. - **Supernode election:** The organizer nodes (laptops at the info booth and entrance) become supernodes. They coordinate traffic between zones and maintain the festival mesh backbone.

Transport mix: Bluetooth for very short-range peer-to-peer, WiFi Direct for local mesh connections, LoRa for ground-to-info-booth links, WebRTC disguised as video call for any internet-bound traffic.

Privacy tier: Attendee phones operate at L1. Organizer laptops operate at L2. LoRa relays operate at L2 if plugged in, L1 if battery-powered.

```
graph TD;
  Attendee1["Attendee (Phone, Leaf)"] -- Bluetooth --> Attendee2["Attendee (Phone, Leaf)"]
  Attendee2 -- WiFi Direct --> InfoBooth["Info Booth (Laptop, Relay)"]
  Attendee3["Attendee (Phone, Leaf)"] -- WiFi Direct --> Stage["Stage (Raspberry Pi, Relay)"]
  Stage -- LoRa --> InfoBooth
  InfoBooth -- LoRa --> SolarRelay["Solar LoRa Relay"]
  InfoBooth -- WebRTC --> Internet["Internet (disguised)"]
  InfoBooth --> Supernode["Supernode (Info Booth)"]
  Stage --> Supernode
```

Lessons from this scenario: - High-density events need relays at multiple points to avoid congestion. - WebRTC traffic disguised as a video call avoids detection and blocking by event

security. - Bluetooth is useful for very short-range connections between nearby attendees.

---

6. Lessons Learned

- **Resilience:** Mesh networks keep working when infrastructure fails. The tiered hierarchy ensures that even if individual nodes fail, the mesh adapts.
- **Privacy:** End-to-end encryption protects users in hostile environments. Privacy tiers automatically adjust based on battery and device capability.
- **Flexibility:** Multiple transports and adapters enable diverse deployments. No single transport is required for the mesh to function.
- **Community:** Local knowledge and training are key to success. The right people in the right roles make the difference.
- **Hierarchy matters:** Assigning roles by device capability, not just availability, produces a more stable and efficient mesh.
- **Battery awareness:** Nodes that ignore battery state degrade the entire mesh. Energy-aware transport selection and privacy tier downgrades are essential.

---

7. Visuals: Scenario Topologies

```
graph TD;
  NodeA["Phone (Leaf)"] -- WiFi Direct --> NodeB["Laptop (Relay)"]
  NodeB -- LoRa --> NodeC["LoRa Relay"]
  NodeC -- LoRa --> NodeD["Supernode"]
  NodeD -- WebRTC --> NodeE["Internet"]
  NodeA -- Bluetooth --> NodeF["Phone (Leaf)"]
  NodeC -- stealth TCP --> NodeG["Gateway"]
```

---

8. Best Practices & Takeaways

- Train users before deployment.
- Test mesh in real-world conditions.
- Use multiple transports for coverage.
- Document lessons and share with the community.
- Assign roles by device capability.
- Monitor battery state and adjust privacy tiers accordingly.
- Elect supernodes early in any deployment.
- Plan for the tiered architecture from the start.
- Cross-reference chapter 17_target_users.md for persona-specific deployment guidance.
- Cross-reference chapter 15_device_hierarchy_and_scaling.md for the full device hierarchy specification.

---

9. Further Reading & Resources

- Mesh Networking 101
- GhostWire Developer Guide
- GhostWire Target Users
- GhostWire Device Hierarchy and Scaling

End of Chapter

Glossary & Reference

Table of Contents

1. Glossary
2. Acronyms & Definitions
3. Quick Reference: Commands & Config
4. Troubleshooting Table
5. Visuals: Reference Flows
6. Cross-References
7. Further Reading & Resources

---

1. Glossary

Term/Acronym	Meaning
Mesh Network	A network where each device (node) relays data for others, creating a resilient web of connections.
Transport	The method or technology used to send messages (Bluetooth, WiFi, LoRa, etc.).
Protocol Adapter	Software that bridges GhostWire to other networks (Briar, Matrix, etc.).
Node	Any device running GhostWire (phone, laptop, server, etc.).
Store & Forward	Technique where messages are cached and relayed when possible.
Sybil Attack	Attack where one entity pretends to be many nodes.
Quota	Limit on messages or actions to prevent abuse.
Federation	Connecting multiple meshes for wider reach.
Blacklist	List of nodes blocked from the mesh.

Term/Acronym	Meaning
QuotaEnforcer	Module that limits message rates.
KeyManager	Module that manages encryption keys.
Supernode	A node elected dynamically based on high uptime, stable IP, and significant bandwidth to act as a relay hub for the mesh.
Relay Hub	A supernode that forwards messages between devices, carrying no trust authority over the data it relays.
Device Tier	A classification of device based on available RAM, storage, and power, determining its role and capabilities in the mesh.
Roles (Full/Relay/Leaf/Sensor/Replicator)	Device roles in GhostWire: Full nodes do everything; Relay nodes forward messages; Leaf nodes send and receive only; Sensor nodes collect data; Replicator nodes store and replicate data.
Battery State	The current charge level of a device, which determines its allowed activity level in the mesh.
Privacy Tier (L0-L3)	A classification of privacy visibility. Servers operate at L3, laptops and tablets at L2, phones and Raspberry Pis at L1. Battery drops push a device one tier down.
Delay-Tolerant Networking (DTN)	A networking approach where messages are stored and forwarded when connectivity is intermittent or unavailable.
PRoPHET	A protocol for delay-tolerant routing that uses encounter history to make forwarding decisions.
Spray-and-Wait	A routing strategy where a message is sent to multiple nodes and the first to reach the destination wins.
LightGBM	A gradient boosting framework used for per-message classification to select the best transport and route in about 77 microseconds on a Raspberry Pi.
Gemma	A family of AI models used in GhostWire for analytical passes and network mapping, running on edge devices and relay laptops.

Term/Acronym	Meaning
Graph Neural Network (GNN)	A neural network that operates on graph structures, used in GhostWire for routing decisions across the mesh.
ML-KEM-768	A post-quantum key encapsulation mechanism providing quantum-resistant key exchange, taking about 890 microseconds on-device.
XChaCha20-Poly1305	An authenticated encryption algorithm used for end-to-end message encryption in GhostWire.
Duress PIN	A PIN that, when entered under coercion, triggers protective actions to hide or wipe sensitive data.
Dead Man's Switch	A mechanism that wipes encryption keys if the user fails to check in within a set time period.
Shamir Secret Sharing	A method to split a secret into parts, requiring a threshold number of parts to reconstruct, used for device recovery.
Decentralized Identifier (DID)	A self-owned identifier that does not depend on a central authority, used for device and user identity in GhostWire.
KERI	A protocol for decentralized identifiers and verifiable events, planned for integration with GhostWire.
SPHINX	A routing protocol that provides unlinkable replies, ensuring that the sender of a message cannot be identified by the recipient.
Unidirectional Reply Route (SURB)	A routing mechanism that allows a recipient to reply to a sender without revealing the sender's identity to the network.
Cover Traffic	Extra network traffic generated to obscure the patterns of real communication, making it harder to analyze who is talking to whom.
Erasure Coding	A method of encoding data into fragments with redundancy, allowing reconstruction from a subset of fragments (e.g., 3 of 5 shards).

Term/Acronym	Meaning
Thompson Sampling	A bandit algorithm used for energy-aware transport selection, balancing exploration of new routes with exploitation of known good ones.
Traffic Armor	A camouflage system that disguises GhostWire traffic and auto-switches transports when one is blocked.
Adapter Registry	A system that allows custom protocol bridges to register at startup, enabling pluggable bridge support.
Knowledge Ark	A distributed storage mechanism for preserving and sharing knowledge across the mesh.
DHT	A distributed hash table used for peer-to-peer storage and lookup of model files and other data across the mesh.

---

2. Acronyms & Definitions

- **E2EE:** End-to-End Encryption
- **UI:** User Interface
- **CLI:** Command-Line Interface
- **API:** Application Programming Interface
- **DoS:** Denial of Service
- **PoW:** Proof of Work
- **PoS:** Proof of Stake
- **LoRa:** Long Range (radio technology)
- **P2P:** Peer-to-Peer
- **PQC:** Post-Quantum Cryptography
- **DTN:** Delay-Tolerant Networking
- **DID:** Decentralized Identifier
- **KERI:** Key Event Receipt Infrastructure
- **SURB:** Unidirectional Reply Route
- **ZKP:** Zero-Knowledge Proof

---

3. Quick Reference: Commands & Config

CLI Commands

```
ghostwire-cli start --transport wifi --mesh mymesh
ghostwire-cli send --to bob --message "Hello, Bob!"
ghostwire-cli nodes
```

Config Example

```
[transports]
bluetooth = true
wifi = true
lora = false
webrtc = true
tcpip = true
stealth_tcp = false
```

---

4. Troubleshooting Table

Problem	Solution
Can't find other nodes	Check transport settings, try another method
Messages not sending	Ensure at least one transport is active
Web UI won't load	Check backend is running, try npm install
LoRa not working	Check hardware, drivers, and permissions
Security warning	Ensure you are using the latest version

---

5. Visuals: Reference Flows

```
graph TD;
  User["User"] --> UI["Web UI/CLI"]
  UI --> Backend["Backend"]
  Backend --> Mesh["Mesh Network"]
  Mesh --> Internet["Internet (optional)"]
```

---

6. Cross-References

- Getting Started
- Architecture Deep Dive
- Transports & Protocols
- Security & Privacy
- Developer Guide
- Device Hierarchy & Power-Aware Scaling
- AI Routing & On-Device Intelligence
- Who GhostWire Serves

---

7. Further Reading & Resources

- GhostWire GitHub
- Mesh Networking 101

End of Chapter

15. Device Hierarchy and Scaling

Table of Contents

1. Executive Summary
2. Why a Hierarchy Exists
3. The Five Device Classes
4. Node Roles Across Four Dimensions
5. The Mailbox Pattern and Leaf Devices
6. Why a Flat Mesh Fails
7. Supernodes and Relay Hubs
8. The Hardware Ladder
9. Battery States and Device Behavior
10. Privacy Tiers by Device
11. Deployment Sizing Benchmarks
12. A Practical Worked Example
13. Best Practices and Anti-Patterns
14. Glossary
15. Further Reading

---

1. Executive Summary

Not every device in a GhostWire mesh is the same. A phone in your pocket has far less memory and battery than the laptop on your desk. GhostWire recognizes this difference and organizes devices into a hierarchy so that the network stays stable, fast, and fair. This chapter explains how devices are classified, what roles they play, how battery life changes their behavior, and how to size a deployment.

The core idea is simple: match each device to the work it can sustain. A phone excels at being a personal endpoint. A server excels at being a backbone. GhostWire lets each device contribute at its own level, and the network as a whole becomes stronger because of that diversity.

---

2. Why a Hierarchy Exists

Plain-Language Explanation

Imagine a convoy of vehicles. At the front, a small motorcycle weaves through traffic, carrying messages between nearby houses. Behind it, a large truck carries supplies for many neighborhoods. The motorcycle is quick but cannot carry as much. The truck is slower but can move far more cargo.

GhostWire works the same way. Your phone is the motorcycle: always with you, always connected, but limited in what it can carry. Your laptop or a nearby server is the truck: more powerful, more memory, and able to relay traffic for many other devices. When the stronger machine is available, it takes on the heavier relay work. When a phone runs low on battery, it hands off its relay duties to a nearby laptop and rests.

This tiered approach keeps the whole convoy moving. Without it, every vehicle would try to do the same job, and the convoy would grind to a halt.

Technical Depth

A flat mesh is one where every device has the same role and the same connections. It sounds fair, but it has a serious problem. As more devices join, the mesh becomes unstable. Every device must track every other device, and the overhead grows faster than the network can handle. In a flat mesh with 100 devices, each device needs to maintain connections and state for all 99 others. The memory and bandwidth requirements grow quadratically.

As the project documentation states, "A flat mesh becomes unstable beyond a certain density. We should introduce a tiered architecture."

GhostWire solves this by introducing tiers. Devices at the top of the hierarchy handle more connections and more data. Devices at the bottom focus on sending and receiving. This tiered approach lets the network grow without collapsing under its own weight.

From the performance scaling documentation, two principles guide relay assignment: "Assign key nodes (with good power and connectivity) as relays" and "Use devices with more RAM/CPU for relays."

---

3. The Five Device Classes

Plain-Language Explanation

GhostWire sorts devices into five classes based on available RAM, storage, and peer capacity. These numbers reflect the real constraints of each device type. A phone has limited memory because it also runs other apps and needs to conserve battery. A tablet has slightly more room for peer connections. A Raspberry Pi has limited RAM but can still function as a gateway. A laptop has enough resources to relay for many devices. A server has the resources to handle many connections.

Understanding these limits helps when planning a deployment. If you try to run a full relay on a phone with 2 GB of RAM, you will run into memory pressure. If you deploy a server as a relay hub, it can handle the load comfortably.

Technical Depth

The five device classes are defined as follows:

Class	RAM Threshold	Max Peers	Storage
Phone	≤ 6 GB	20	1 GB
Tablet	≤ 8 GB	30	2 GB
Raspberry Pi	≤ 4 GB	15	500 MB

Class	RAM Threshold	Max Peers	Storage
Laptop	$\leq$ 16 GB	50	5 GB
Server	>16 GB	100+	50 GB+

These limits are drawn from the GhostWire Device Tiers Guide. The RAM threshold determines the device class automatically. The max peers limit controls how many other devices a single node can maintain connections with. The storage limit caps how much mesh data each device can hold locally.

The Raspberry Pi is a special case. Even though it has 4 GB of RAM, it is detected as a Raspberry Pi rather than a Laptop because the detection logic checks for the Raspberry Pi hardware signature first.

---

4. Node Roles Across Four Dimensions

Plain-Language Explanation

Every device in GhostWire can take on one or more roles. These roles are dynamic, meaning they can change as conditions shift. A phone that starts as a leaf node can become a relay when it plugs in and its battery is full. GhostWire defines five roles: Full, Relay, Leaf, Sensor, and Replicator. Each role is described across four dimensions: forwarding duty, power behavior, storage responsibility, and dialability.

Technical Depth

The role definitions are drawn from the GhostWire External Project Learnings, which studied patterns from Meshtastic, Berty, and Briar to build a device capability model:

Role	Forwarding	Power	Storage	Dialable
Full	Full DHT + gossip	Always on	Full node store	Yes (listeners)
Relay	Core ports only	Low-current/solar	Warm tier	Yes (relay reservation)
Leaf	Local only (never relays)	Sleep-heavy	Minimal (own groups only)	No (outbound only)
Sensor	Local only	Ultra-low (duty cycle)	Read-only	No
Replicator	Forward encrypted blocks	Always on	Full (cannot decrypt)	Yes

A Leaf node is the simplest role. It can send and receive messages but does not relay for others. Think of it as the endpoint of a line. A phone on the move is often a leaf node.

A Relay node forwards messages between other devices. It keeps the paths open. Relays are the backbone of the network, and they are assigned to devices with good power and connectivity. Relay nodes are often low-current or solar-powered devices.

A Full node stores a complete copy of the mesh data and can relay messages for any other node. Full nodes are typically laptops or servers with enough RAM and storage to hold the entire network state.

A Sensor node collects and publishes data from the physical world, such as temperature readings or location markers. Sensors typically have limited bandwidth and send small packets at regular intervals.

A Replicator node stores copies of important data to ensure it survives even if other nodes go offline. A replicator might store a copy of the mesh routing table or a set of important community messages.

Critical Honesty About Relay Authority

It is important to understand what relays and supernodes can and cannot do. Relays and supernodes carry no trust authority. They route messages, but they cannot read the contents of those messages or override any decisions. The encryption end-to-end means that only the sender and the intended recipient can read a message. A relay is like a postal worker who carries a sealed letter but cannot open it. No relay, no supernode, and no Relay Hub can ever read, modify, or block a message. They are pure conduits.

---

5. The Mailbox Pattern and Leaf Devices

Plain-Language Explanation

What happens when a leaf device needs to receive a message but is turned off or out of range? GhostWire uses a mailbox pattern. An always-on external buffer holds incoming messages until the leaf device reconnects. This is like a post office box: the mail waits for you, even when you are not home.

The core insight is captured in a simple phrase from the Briar project's design: "power-constrained nodes offload storage+forwarding to a 'friend.' " When your phone is low on battery, it asks a trusted nearby device to hold its messages and forward them on its behalf.

Technical Depth

In GhostWire, the mailbox is implemented as a designated node that accepts messages destined for offline leaf devices. When a leaf device reconnects, it pulls its queued messages from the mailbox.

---

6. Why a Flat Mesh Fails

Plain-Language Explanation

You might wonder: why not just let every device connect to every other device? That would be the simplest approach. But it does not work beyond a certain point. As more devices join, the overhead grows faster than the network can handle. The network becomes unstable.

Technical Depth

The scalability roadmap captures this problem directly: "A flat mesh becomes unstable beyond a certain density. We should introduce a tiered architecture."

In a flat mesh, every device must maintain a connection to every other device. The number of connections grows quadratically with the number of devices. A mesh with 10 devices needs about 45 connections. A mesh with 100 devices needs about 4,950 connections. A mesh with 1,000 devices needs about 499,500 connections. No consumer device can handle that many connections.

The solution is hierarchical organization. Instead of every device connecting to every other device, devices organize into tiers. Leaf devices connect to relay nodes. Relay nodes connect to supernodes. Supernodes connect to the global backbone. This reduces the number of connections each device needs to maintain and lets the network scale.

The scalability targets from the project roadmap show this progression: the current phase supports 10 to 50 nodes. The V2.2 target is 100 to 500 nodes with Protobuf and Kademlia DHT optimization. The V3.0 target is 1,000 or more nodes with hierarchical supernodes and regional sharding. The V4.0 target is 10,000 or more nodes.

---

7. Supernodes and Relay Hubs

Plain-Language Explanation

In larger deployments, GhostWire uses supernodes, also called Relay Hubs. These are devices that have high uptime, stable IP addresses, and significant bandwidth. They act as the backbone of the mesh, connecting smaller clusters together. The election process is dynamic. If a Relay Hub goes offline, the network detects this and elects a new one from the available nodes.

Technical Depth

The supernode election process is described in the scalability roadmap: "Supernode Election: Dynamically elect nodes with high uptime, stable IP addresses, and significant bandwidth to act as 'Relay Hubs.' "

The election criteria are straightforward. High uptime means the node is online most of the time. A stable IP address means the node can be found consistently. Significant bandwidth means the node can handle the traffic load of relaying for many other devices.

Regional DHT sharding is a related concept. Instead of one global Kademlia DHT, the network uses regional shards. Each shard handles a geographic area, and a global backbone connects the shards together. This means that a device in one city does not need to track every peer in another city.

The hierarchy of the network can be visualized as follows:

```
graph TD;
    Phone["Phone / Leaf"] -->|Sends/Receives| Relay["Relay Node"]
    Relay -->|Forwards| Supernode["Supernode / Relay Hub"]
    Supernode -->|Connects| ShardA["Regional DHT Shard A"]
    Supernode -->|Connects| ShardB["Regional DHT Shard B"]
    Supernode -->|Backbone| Global["Global Backbone"]
```

```
Phone2["Phone / Leaf"] --> Relay
Phone3["Phone / Leaf"] --> Relay
Relay2["Relay Node"] --> Supernode
```

This diagram shows the hierarchy from phone/leaf devices at the bottom, through relay nodes in the middle, to supernodes and the global backbone at the top.

---

8. The Hardware Ladder

Plain-Language Explanation

GhostWire runs on a wide range of hardware, from a $20 ESP32 microcontroller to a full server-class machine. The hardware ladder shows how each level fits into the network.

Technical Depth

The hardware ladder has three main rungs:

The $20 ESP32. The ESP32 is a tiny microcontroller with about 148KB of free RAM when WiFi and BLE are active. It cannot run the full libp2p stack that larger devices use. The reason is straightforward: rust-libp2p requires a runtime of several megabytes, and the ESP32 simply does not have that much memory available. Instead, the ESP32 uses a gateway-bridged architecture. The ESP32 forms a mini-mesh using ESP-NOW or Bluetooth Low Energy, and it bridges that mini-mesh to the wider GhostWire network over a WiFi uplink. This design means that even a $20 device can participate in a GhostWire mesh, but it does so as a leaf or sensor node, not as a relay. The gateway handles the heavier routing and relay duties.

The $100 Raspberry Pi. A Raspberry Pi has more memory and can run a fuller version of GhostWire. It serves as a gateway node for a neighborhood or a small deployment. It can relay messages, store data, and act as a mailbox for leaf devices.

Full Server-Class Nodes. Servers have the most resources and can handle the heaviest loads. They act as supernodes, relay hubs, and replicators. They store the full mesh state and serve as the backbone for large deployments.

The gateway-bridged architecture for ESP32 devices follows a proven pattern from the Meshtastic project, which uses a LoRa mesh plus a bridge process, and from the microfips project, which runs no_std cryptographic handshakes on constrained hardware. The crypto layer works on ESP32; the blocker is the libp2p machinery.

---

9. Battery States and Device Behavior

Plain-Language Explanation

Battery level is one of the most important factors in how GhostWire behaves. The network adjusts automatically based on how much charge a device has left. There are four battery states that determine what a device can do. When a device is charging, it has no restrictions. When the battery is above 50 percent, the device operates normally. When the battery is between 20 and 50 percent,

the device reduces its cover traffic by 30 percent. When the battery is at 20 percent or below, the device drops to the lowest privacy tier and stops relaying.

Technical Depth

The four battery states are defined as follows:

State	Condition	Behavior
Charging	battery_status == Charging	Unrestricted. Full relay, storage, and participation.
Above 50%	battery_level > 50%	Normal operation. All features available.
20% to 50%	20% < battery_level <= 50%	Reduced. Cover traffic cut by 30%.
20% or below	battery_level <= 20%	Minimal. Drop to privacy L0/L1. Stop relaying.

This means a phone with a nearly empty battery stops acting as a relay. The transition is smooth.

OEM Battery Considerations

Many devices used in East Africa and Kenya run on Android phones from manufacturers like Samsung, Xiaomi, Huawei, OPPO, Vivo, and Transsion. These devices often have aggressive battery management that kills background apps to save power. GhostWire follows the guidance of the dontkillmyapp project to request that the operating system keep the app running in the background.

This is particularly important for disaster responders and rural community members who may rely on their phones as mesh nodes throughout the day. Without the battery optimization exemption, the GhostWire app might be killed by the operating system, breaking the mesh connection unexpectedly. The OEM battery guide includes specific detection methods for each manufacturer, including Transsion, which uses a basic battery reporting interface at `/sys/class/power\_supply/battery/`.

---

10. Privacy Tiers by Device

Plain-Language Explanation

GhostWire assigns a privacy tier to each device based on its class and current battery state. The privacy tier determines how much metadata the device broadcasts and what relay permissions it has. A higher privacy tier means the device can broadcast more metadata and participate more fully in the mesh. A lower privacy tier means the device is more constrained.

Technical Depth

The default privacy tiers by device class are:

Device Class	Default Privacy Tier
Server	L3
Laptop	L2
Tablet	L2
Phone	L1
Raspberry Pi	L1

These defaults reflect a balance between privacy and practicality. A server is always plugged in and can afford maximum privacy settings. A phone is battery-powered and starts at a lower tier.

The privacy tiers themselves are defined as follows:

Level	Name	Latency	Cover Traffic	Use Case
L0	None	0ms	0%	Debugging, trusted LAN
L1	Low	50ms	10%	Daily messaging, good network
L2	Medium	150ms	30%	Sensitive communication
L3	High	500ms	50%	High-risk, censorship zones

When a device's battery drops below 20 percent, it drops one privacy tier. A laptop at 15 percent battery behaves like a phone at full charge. The lower tier reduces metadata broadcasts and limits relay activity.

The battery adaptation rule is simple: charging or above 50 percent means no change. Between 20 and 50 percent means downgrade one level. Below 20 percent means downgrade to L0 or L1.

---

11. Deployment Sizing Benchmarks

Plain-Language Explanation

The right deployment size depends on the scenario. A home mesh with five devices behaves very differently from a community mesh with fifty devices or an enterprise mesh with two hundred or more. The benchmarks below give a starting point for planning.

Technical Depth

The sizing benchmarks are drawn from the performance scaling documentation:

Scenario	Devices	Avg. Latency	Max Throughput
Home	5	50ms	1 Mbps
Community	50	100ms	5 Mbps
Disaster Field	20	200ms	500 Kbps
Enterprise	200+	150ms	10 Mbps

Key guidance for sizing includes: assign key nodes with good power and connectivity as relays, use devices with more RAM and CPU for relays, plan for regional DHT sharding when the mesh exceeds fifty devices, and monitor battery levels so the network can adjust automatically.

---

12. A Practical Worked Example

Plain-Language Explanation

Let us walk through a realistic scenario. Imagine a small community center where four devices are part of the same GhostWire mesh: a phone, a nearby laptop, a Raspberry Pi, and a server.

The phone belongs to a community member who walks around the neighborhood. It acts as a leaf node, sending and receiving messages but not relaying for others. The laptop sits on a desk and acts as a relay node. The Raspberry Pi is mounted near the entrance and acts as a gateway node. The server is a full machine in a back room that acts as a supernode and relay hub.

When the phone is in the neighborhood with low battery, it drops to leaf mode and relies on the laptop and Raspberry Pi to relay its messages. The server handles the heavy lifting of connecting to the global backbone.

Technical Depth

This worked example illustrates the four-dimensional role assignment in practice. The phone is a Leaf with local-only forwarding, sleep-heavy power behavior, minimal storage, and outbound-only dialability. The laptop is a Relay with core-port forwarding, normal power behavior, warm-tier storage, and relay reservation dialability. The Raspberry Pi is a Relay with similar characteristics but lower peer capacity. The server is a Full node with full DHT and gossip forwarding, always-on power, full node storage, and listener dialability.

The privacy tiers also adapt in this scenario. The server runs at L3 with 50 percent cover traffic and 500ms mixing delay. The laptop and Raspberry Pi run at L2 with 30 percent cover traffic and 150ms mixing delay. The phone runs at L1 with 10 percent cover traffic and 50ms mixing delay. If the phone's battery drops below 20 percent, it drops to L0 or L1 and stops relaying entirely.

The mailbox pattern is also in play. When the phone leaves the community center, the laptop holds any messages destined for the phone. When the phone returns, it pulls those messages from the laptop's mailbox buffer.

---

13. Best Practices and Anti-Patterns

Plain-Language Explanation

- Use the tiered architecture. Do not try to make every device do the same job.
- Assign relay roles to devices with stable power and good connectivity.
- Monitor battery levels and let the network adjust automatically.
- Use the mailbox pattern for leaf devices that need to receive messages offline.
- Plan for regional DHT sharding in deployments larger than fifty devices.
- Enable battery optimization exemptions on Android devices from manufacturers like Samsung, Xiaomi, and Huawei.
- Test your deployment at the scale you intend to operate.
- Use the hardware ladder to set expectations. A $20 ESP32 is a leaf node, not a relay.

- Remember that relays carry no trust authority. They forward messages but cannot read or censor them.

Anti-Patterns

- Trying to run a full relay on a phone with a low battery.
- Ignoring the hardware ladder and expecting a $20 ESP32 to do the work of a server.
- Assuming relays can read or control message content. They cannot. Relays carry no trust authority.
- Deploying a flat mesh beyond its stable density limit.
- Forgetting to configure the dontkillmyapp exemption on Android devices.
- Overlooking battery management on OEM Android phones, which may kill the GhostWire app in the background.
- Expecting an external security audit to be complete. The audit is planned, not done. Organizations such as Cure53, NCC Group, and Least Authority have been identified as potential auditors, but no audit has been completed yet.

---

14. Glossary

- **Device Class:** A grouping of devices by RAM, storage, and peer capacity (Phone, Tablet, Raspberry Pi, Laptop, Server).
- **Relay Hub:** A supernode with high uptime and bandwidth that connects mesh segments.
- **Mailbox Pattern:** An always-on external buffer that holds messages for offline leaf devices.
- **Privacy Tier:** A level of metadata exposure and relay permission assigned to a device based on its class and battery state.
- **Regional DHT Sharding:** Splitting the distributed hash table into geographic segments for scalability.
- **Gateway-Bridged Architecture:** A design where a small device uses a short-range mini-mesh and bridges to the wider network via a WiFi uplink.
- **Full Node:** A device that stores the complete mesh data and can relay for any other node.
- **Leaf Node:** A device that sends and receives messages but does not relay for others.
- **Replicator Node:** A device that stores copies of important data for disaster recovery.
- **Trust Authority:** The ability to read, modify, or block messages. Relays and supernodes have no trust authority.
- **Cover Traffic:** Fake or padding messages sent to obscure the timing and size of real messages.
- **Supernode Election:** The dynamic process of selecting nodes with high uptime, stable IP addresses, and significant bandwidth to act as Relay Hubs.

---

15. Further Reading

- GhostWire Project Overview
- Performance & Scaling
- Architecture Deep Dive
- Transports & Protocols
- Deployment & Operations
- Case Studies & Real-World Stories

- GhostWire Device Tiers Guide
- GhostWire Privacy Tiers Guide
- GhostWire Scalability Roadmap
- External Project Learnings
- dontkillmyapp Project

End of Chapter

16. AI Routing and Intelligence

Table of Contents

1. Executive Summary
2. The Four-Layer AI Routing Stack
3. Layer 1: LightGBM Per-Message Classifier
4. Layer 2: Gemma 4 Edge Model
5. Layer 3: Graph Neural Network (GNN)
6. Layer 4: PRoPHET Delay-Tolerant Routing
7. Model Ladder per Hardware
8. Model Update Flow and Training Pipeline
9. Thompson Sampling Transport Selection
10. Privacy and Local AI Decisions
11. Knowledge Ark and Anomaly Detection
12. Worked Example: A Message's Journey
13. Future Scopes
14. End of Chapter

---

1. Executive Summary

GhostWire does not rely on a central server to decide where your messages go. Instead, every device runs a four-layer AI routing stack locally. The stack reads the mesh network, predicts the best path, and adapts in real time. A lightweight model makes the fastest decisions. A larger model provides deeper analysis when needed. A graph neural network optimizes across the whole topology. And a battle-tested delay-tolerant protocol stands ready as the final fallback.

All of this happens on your own device. No cloud AI dependency. No external server making routing choices for you. Your data stays on your machine, and any telemetry shared upward is strictly opt-in and anonymized.

This chapter walks through each layer, the hardware each model runs on, how models get updated, how transport selection works, and what the future holds.

---

2. The Four-Layer AI Routing Stack

The AI routing system is a cascading fallback chain. Each layer handles a different kind of decision. If one layer is unavailable, the next takes over. The system always has a working answer.

```
graph TD;
    subgraph L1["Layer 1: LightGBM"]
        L1_In["2,387 features"] --> L1_Model["LightGBM ONNX"]
        L1_Model --> L1_Out["77$\mu$s inference"]
    end
    subgraph L2["Layer 2: Gemma 4"]
        L2_In["Mesh state"] --> L2_Model["Gemma 4 270M"]
        L2_Model --> L2_Out["~30s reasoning"]
    end
    subgraph L3["Layer 3: GNN"]
        L3_In["Graph topology"] --> L3_Model["GAT 4-layer 8-head"]
        L3_Model --> L3_Out["91$\mu$s prediction"]
    end
    subgraph L4["Layer 4: PRoPHET"]
        L4_In["Encounter history"] --> L4_Model["Epidemic routing"]
        L4_Model --> L4_Out["<0.01ms fallback"]
    end
    L1_Out -.->|"fallback"| L2_In
    L2_Out -.->|"fallback"| L3_In
    L3_Out -.->|"fallback"| L4_In
    L4_Out --> Route["Selected Route"]
```

The arrows above show the fallback direction. Layer 1 is the fast path. If it fails, the system drops to Layer 2. If Layer 2 fails, Layer 3 takes over. Layer 4 is always available because it requires no model file at all.

---

3. Layer 1: LightGBM Per-Message Classifier

Plain-Language

Layer 1 is the fastest decision-maker in the stack. It looks at 2,387 features about the mesh network and outputs a delivery probability score in under a millisecond. Think of it as the reflex system: it reacts instantly, before you even think about the problem.

The model was trained on real data from the GuifiSants mesh network in Catalonia. That means it has seen how actual nodes behave, not just how simulated nodes behave. The training set included 7,931 samples across 63 nodes over 31 days.

Technical Depth

The model is a LightGBM gradient boosting decision tree ensemble exported to ONNX format. It takes a single 2,387-dimensional feature vector and outputs a score between 0.0 and 1.0 representing the predicted probability of successful delivery.

Property	Value
Algorithm	LightGBM (Gradient Boosting Decision Trees)
Format	ONNX (Open Neural Network Exchange)
Input dimensions	2,387 features
Output	Delivery probability score (0.0 to 1.0)
Inference time	~77μs on Raspberry Pi
Model size	~2-5 MB

The feature vector is built from seven categories:

Category	Feature Count	Examples
Link quality	256	SNR, RSSI, packet loss rate, retransmission count
Latency	128	RTT, jitter, one-way delay, queue depth
Node health	64	Uptime, CPU usage, memory usage, buffer occupancy
Topology	512	Degree, betweenness centrality, clustering coefficient
Temporal	256	Hour of day, day of week, season, activity patterns
Geographic	128	Distance, elevation, line-of-sight probability
Historical	512	Rolling averages over 1h, 6h, 24h, 7d
Derived	531	Interaction terms, polynomial features, lag features

Missing features are filled with the median value from the training distribution. If more than 30% of features are missing, Layer 1 is considered unavailable and the system falls back to Layer 2.

---

4. Layer 2: Gemma 4 Edge Model

Plain-Language

When the fast path is not enough, the system turns to Gemma 4. This is a language model that actually understands mesh topology. It reads the network state, thinks about the options, and gives a reasoned recommendation. It runs every 30 seconds on a schedule, not on every single message, because it is slower and more expensive than Layer 1.

The model runs entirely on your device through Ollama or LiteRT. No data leaves your machine. The model is quantized to 4-bit precision (Q4_K_M) to keep it small enough for edge hardware.

Technical Depth

Property	Value
Model	Gemma 4 (270M parameter variant)
Context window	32,768 tokens
Runtime	Ollama or LiteRT
Quantization	Q4_K_M (4-bit, ~150 MB)
Inference time	~30 seconds
Hardware minimum	4 GB RAM

The prompt given to Gemma 4 includes the full network state: available peers, link qualities, recent delivery rates, current time, message priority, and TTL. The model returns a route as an ordered list of peer IDs, a confidence score, and a text explanation.

On devices with less than 4 GB of RAM, Layer 2 is skipped entirely and the system falls straight to Layer 3.

---

5. Layer 3: Graph Neural Network (GNN)

Plain-Language

Layer 3 looks at the mesh as a graph. Every node is a point. Every connection between nodes is a line. The neural network learns patterns across this graph structure to predict which paths will work best. It is fast and precise, running in under a hundred microseconds.

Technical Depth

Property	Value
Architecture	Graph Attention Network (GAT)
Layers	4 GAT layers
Attention heads	8 heads per layer
Format	ONNX
Inference time	91μs (ONNX Runtime, CPU)

The mesh network is represented as a graph G = (V, E). Nodes are active peers. Edges are active communication links. Each node has features like uptime, buffer usage, average latency, packet loss rate, trust score, geographic coordinates, and transport capacity. Each edge has features like link quality, round-trip time, bandwidth, and how recently the peers were seen.

The GAT layers compute attention-weighted messages. Each node pays attention to its neighbors, weighting their importance based on learned patterns. Eight attention heads run in parallel, and their results are concatenated. The output is a delivery probability score for each candidate route.

---

6. Layer 4: PRoPHET Delay-Tolerant Routing

Plain-Language

Layer 4 is the oldest and most reliable layer. It does not need any AI model. It does not need any model file at all. It uses a simple formula based on how often nodes have met each other. If node A has met node B many times, and node B has met node C many times, then A has a good chance of reaching C through B. This is called transitivity.

PRoPHET is always on. It is the safety net that catches every message when all the AI layers fail.

Technical Depth

Property	Value
Protocol	PRoPHET v2 (epidemic routing)
Delivery predictability update	P(a,b) = P_old + (1 - P_old) × P_init
Time decay	$\gamma = 0.98$ per hour
Transitivity weight	$\beta = 0.25$
Initial predictability	P_init = 0.75

When node A encounters node B, the predictability P(a,b) increases. Over time, if they do not meet, the predictability decays. When A meets B, A also learns about everyone B has met, and updates its predictability to those nodes through transitivity.

The routing decision is simple: forward the message to the neighbor with the highest predicted chance of reaching the destination. Ties are broken by buffer usage, recency of encounter, and finally by a deterministic hash of the bundle ID.

PRoPHET works alongside Spray-and-Wait for initial distribution. Spray-and-Wait sends up to a configurable number of copies (default 3, maximum 10) to encountered peers, then waits for one of them to reach the destination. GhostWire uses predictive spray by default, which combines Spray-and-Wait with PRoPHET predictability scores.

SyncBloom is a 1024-bit probabilistic data structure used for bundle deduplication during peer synchronization. When two peers meet, they exchange SyncBlooms to discover which bundles the other is missing, without revealing bundle contents.

---

7. Model Ladder per Hardware

Different hardware gets different sized models. A tiny phone runs a small model. A powerful server runs a large model. The ladder ensures every device uses the best model its hardware can handle.

Model Variant	Hardware	RAM/VRAM	Latency
Gemma 4 E2B	Edge devices, phones, Raspberry Pi	8 GB RAM	~100ms
Gemma 4 E4B	Relay laptops	10 GB VRAM	~50ms
Gemma 4 26B MoE	Coordinator nodes	20 GB RAM	~80ms

Model Variant	Hardware	RAM/VRAM	Latency
Gemma 4 31B	Central servers	24 GB VRAM	~120ms

The E2B variant is the smallest at 2 billion parameters. It runs on 8 GB of RAM and is ideal for Raspberry Pi mesh nodes and phones. The E4B variant at 4 billion parameters targets relay laptops with mid-range GPUs. The 26B MoE (Mixture of Experts) model activates only 4 billion parameters at a time, giving flagship quality at edge speeds. The 31B dense model runs on central servers with the most powerful GPUs.

All models run through Ollama or LiteRT on the device. LiteRT provides native on-device inference without requiring Ollama. The models are downloaded once and cached locally.

---

8. Model Update Flow and Training Pipeline

Plain-Language

Models get smarter over time, but they do not learn from your private data. Here is how it works: participating nodes (only if you opt in) send anonymized telemetry summaries upward. Those summaries go to training machines. The training machines use the data to build better models. The finished model files are then distributed back down to your device. Your raw data never leaves your machine. No gradients are ever shared. This is not federated learning.

Technical Depth

The model update pipeline follows these steps:

1. Collect telemetry from participating nodes (opt-in only)
2. Preprocess: normalize, handle missing values, feature engineering
3. Train LightGBM on labeled data (delivery success or failure)
4. Export to ONNX format
5. Train GNN on graph-structured data
6. Export to ONNX format
7. Publish model hashes to the DHT
8. Distribute via peer-to-peer sync

Model verification is strict. Before loading a new model, the system computes a BLAKE3 hash of the model file and compares it against the expected hash published by the GhostWire maintainers. If the hash does not match, the model is rejected and a warning is logged.

The key distinction: this is not federated learning. No gradients are aggregated across nodes. Raw training data never leaves a node. Only an opt-in telemetry summary goes up, and only finished model files come down. The model files are distributed through DHT hash publication, where peers verify against known-good hashes, plus peer-to-peer sync. In-app updates check for new models weekly.

---

9. Thompson Sampling Transport Selection

Plain-Language

Once the AI decides which route to take, the system still needs to pick the physical transport: Bluetooth, WiFi, LoRa, Reticulum, or TCP/QUIC. This is done with Thompson Sampling, a method that balances exploration (trying new transports) with exploitation (sticking with what works). Each transport has an energy efficiency weight that reflects how much battery it drains.

Technical Depth

The transport selector uses a `TransportMetrics` struct with fields for signal strength, throughput, packet loss, latency, energy efficiency, congestion, and reliability. Each transport type has a weight profile that encodes design intent. The energy efficiency weights are:

Transport	Energy Efficiency Weight
BLE	0.30
LoRa	0.35
Reticulum	0.25
WiFi	0.05
TCP/QUIC	0.0

BLE and LoRa score highest on energy efficiency because they are designed for low-power communication. WiFi uses moderate energy. TCP/QUIC scores zero because it assumes a powered, connected device.

The scoring blends a weighted sum of normalized metrics (70%) with Thompson Sampling exploration (30%). A Beta distribution posterior tracks each transport's success rate in each context (battery tier, signal tier, congestion tier). Hysteresis prevents rapid switching: a new transport must beat the current one by at least 0.1 score points, and a 30-second cooldown applies between switches.

Black-hole detection monitors each transport's success rate. If a transport fails too many times in a sliding window, it is blocked until a success resets the detector.

---

10. Privacy and Local AI Decisions

Plain-Language

Every AI decision in GhostWire happens on your device. The models run locally. The routing choices are made locally. No cloud server sees your network state, your message content, or your routing decisions.

Telemetry is strictly opt-in. If you choose to participate, only anonymized summary statistics leave your device. Node IDs are hashed before export. No personally identifiable information is included in any training data.

There is no cloud AI dependency for core routing. The system works entirely offline. If you are in a remote area with no internet connection, the AI routing still functions because every layer runs on your own hardware.

---

11. Knowledge Ark and Anomaly Detection

Plain-Language

Beyond routing, GhostWire has a knowledge layer called the Knowledge Ark. Every node can store notes, articles, code snippets, and references. These entries replicate across trusted peers through the DHT. The Knowledge Ark is content-addressed, meaning every entry has a unique hash that verifies its integrity.

Anomaly detection runs alongside the routing stack. An Isolation Forest model monitors mesh network traffic and flags unusual patterns. This is the early warning system: if a node is degrading or behaving strangely, the anomaly detector catches it before it causes routing problems.

Technical Depth

The Knowledge Ark (`knowledge\_ark.rs`) manages collections of entries. Each entry has a title, body, kind (note, article, code, reference, dataset, media clip, question, or answer), access level (public, trusted, shared, or private), and content-addressed ID computed as a BLAKE3 hash. Entries are stored in a local sled tree for search and retrieval. Full-text search indexes words across titles and bodies.

The Isolation Forest model (`anomaly\_detector.rs`) is an ensemble of decision trees. Each tree partitions the feature space randomly. Anomalies have shorter average path lengths through the trees. The model loads from a JSON file and computes an anomaly score for each feature vector. A positive score means normal; a negative score means anomalous. The model uses 200 trees across 39 features.

---

12. Worked Example: A Message's Journey

Plain-Language

Imagine you are on your phone, sending a message to a friend who is two neighborhoods away. Your phone has a nearby laptop acting as a relay. Here is what happens, step by step, as your message passes through all four AI layers.

Technical Depth

Step 1: Layer 1 (LightGBM, ~77μs)

Your phone extracts 2,387 features from the mesh telemetry. The feature vector includes your phone's RSSI (-45 dBm), the laptop relay's latency (18 ms), packet loss (2%), and the trust score of the relay peer (0.9). The LightGBM ONNX model processes this vector in 77 microseconds and outputs a delivery probability of 0.94. Layer 1 succeeds. The system has a candidate route through the laptop relay.

Step 2: Layer 2 (Gemma 4, ~30 seconds)

On a 30-second schedule, the Gemma 4 model runs a deeper analysis. The prompt includes the full network state: your phone, the laptop relay, three other nearby peers, their link qualities, and

the destination's last-seen timestamp. Gemma 4 returns a route through the laptop relay with a confidence score of 0.91 and a text explanation noting that the relay has the strongest link quality and lowest latency to the destination.

Step 3: Layer 3 (GNN, 91μs)

The Graph Attention Network constructs the mesh as a graph. Your phone is a node. The laptop relay is a connected node. The destination is a distant node. The GAT layers compute attention weights across the graph. The GNN outputs a delivery probability of 0.93 for the route through the laptop relay, confirming Layer 1 and Layer 2.

Step 4: Layer 4 (PRoPHET, <0.01ms)

Even though Layers 1 through 3 all agreed, Layer 4 runs its PRoPHET formula as a safety check. The delivery predictability from your phone to the laptop relay is 0.88 (based on recent encounter history). The predictability from the laptop relay to the destination is 0.82. The transitivity boost gives a combined predictability of 0.85. Layer 4 confirms the route.

Transport Selection

The transport selector runs Thompson Sampling. Your phone has BLE and WiFi available. The energy efficiency weights favor BLE (0.30) over WiFi (0.05). The Thompson Sampling posterior for BLE in the current context (high battery, strong signal, low congestion) returns a score of 0.82. WiFi returns 0.71. BLE wins. The message is sent over Bluetooth Low Energy to the nearby laptop relay.

The Relay Forward

The laptop relay receives the bundle over BLE. It runs its own four-layer stack. Its Gemma 4 E4B model (4 billion parameters, 10 GB VRAM) analyzes the mesh. Its GNN confirms the next hop toward the destination. It forwards the bundle over WiFi or LoRa, depending on the transport selector's decision at that moment.

Bundle Details

The bundle contains the encrypted payload (XChaCha20-Poly1305), a 32-byte BLAKE3 bundle ID, a TTL of 12 hours (Normal priority), a SyncBloom for deduplication, and the previous hop list for PRoPHET state updates. If the laptop relay loses connectivity, the bundle sits in its 50 MB buffer until the next encounter.

---

13. Future Scopes

Plain-Language

GhostWire has four major directions for the future. Each one pushes the system further toward a truly resilient, private, and accessible mesh network.

Technical Depth

Nucleus ESP32: The goal is to shrink GhostWire to run on $1.00 microcontrollers like the ESP32 and STM32. The current system needs about 50 MB of RAM. The Nucleus proposal strips `libp2p` and `tokio` in favor of `no\_std` alternatives like `embassy` and `smol`. A minimal Gossip Light protocol

would run on low-power radio links. A bridge would connect Nucleus nodes (sensors, IoT devices) to Core nodes (hubs, servers). This would make GhostWire the global standard for $10 off-grid mesh devices.

Sovereign Persona ZKP: Identity would move beyond simple keys to rich, decentralized profiles. Decentralized Identifiers (DIDs) would follow W3C standards. Zero-Knowledge Proofs would allow attribute verification without revealing identity, such as proving you are a verified rescue worker without disclosing your name. Social recovery mechanisms would let a quorum of trusted peers help restore a lost identity.

Recursive DHT: The DHT would scale from 100 nodes to 100,000+ nodes. S/Kademlia hardening would protect against Sybil and Eclipse attacks. Hierarchical sharding would divide the global mesh into regional shards to reduce routing table bloat on mobile devices. Supernode orchestration would dynamically promote high-uptime, high-bandwidth nodes to act as regional gateways.

PQC Armor: Post-quantum cryptography would protect GhostWire traffic against future quantum computer decryption. Hybrid ML-KEM (Kyber, NIST standard) would integrate alongside X25519. PQ-Signatures would transition identity signing from Ed25519 to Dilithium (ML-DSA). Algorithm agility would let nodes step down to legacy crypto only if the peer does not support PQC.

---

14. End of Chapter

End of Chapter

17. Target Users

Table of Contents

1. Executive Summary
2. The Six Target Personas
3. Persona-to-Capability Map
4. Four Illustrative Scenarios
5. Who Is Behind GhostWire
6. Which Persona Should Read Which Chapter
7. End of Chapter

Executive Summary

GhostWire is built for six communities that need communication when traditional networks fail, when they are watched, or when no network exists at all. This chapter describes each community, what they need, and how GhostWire features map to those needs. If you are not a technical reader, this chapter will help you understand who GhostWire is for and why it matters.

The six communities are disaster responders who coordinate when cell towers fall, privacy advocates who refuse to be tracked, rural communities that lack reliable internet, activists and journalists who speak under threat, whistleblowers who expose wrongdoing without being traced, and developers and researchers who build on open code. Each community uses a different combination of GhostWire features, and the modular design of the platform means that no one is forced to use features they do not need.

The project is built by Team GhostWire, seven members from the Open University of Kenya, and was submitted to the Global Challenge Design for Future 2026 competition. The team aligns its work with the United Nations Sustainable Development Goals 9, 11, and 17. The project carries a simple belief: the internet was built to be ungovernable, and communication infrastructure should be restored to the people, not owned by corporations. This chapter provides a plain-language overview. The technical details live in the chapters that follow.

This chapter is organized as follows. Section 1 describes each persona in detail with their tagline and the features that serve them. Section 2 provides a visual map of which capabilities serve which persona. Section 3 presents four illustrative scenarios drawn from the Case Studies chapter. Section 4 introduces the team behind the project. Section 5 provides a routing table so each reader can find the chapters most relevant to their interests.

---

1. The Six Target Personas

GhostWire serves six distinct communities. Each has a different reason for needing the platform, a different threat model, and a different set of features that matter most to them. The taglines below are drawn directly from the project website and describe each community in a single sentence.

1. Disaster Responders

Tagline: “Coordinate when cell towers fail. Works on battery and solar”

Disaster responders need to communicate when the infrastructure is destroyed. Cell towers go down, internet connections disappear, and traditional phones become useless. GhostWire provides WiFi Direct and Bluetooth Low Energy mesh networking, LoRa for long-range communication, and delay-tolerant delivery so that messages are stored and forwarded when a connection becomes available.

In a hurricane response scenario within 200 kilometers of infrastructure, responders can deploy LoRa nodes at key locations and use battery-powered devices to maintain coordination. Messages hop from node to node until they reach their destination, even if no direct path exists. The store-and-forward routing ensures that no message is lost, even when connections are intermittent. This is covered in detail in Chapter 04 on transports.

Disaster responders also benefit from the security architecture described in Chapter 05. AES-256-GCM encryption ensures that sensitive operational data stays private, and the delay-tolerant networking layer ensures that messages survive temporary disconnections. The device hierarchy in Chapter 15 describes how responders can use battery-aware scaling to preserve both charge and operational security.

2. Privacy Advocates

Tagline: "No logs. No traces"

Privacy advocates need communication that leaves no metadata behind. GhostWire supports metadata minimization techniques and multi-profile identities, allowing users to maintain separate personas for different contexts. The network does not require phone numbers, email addresses, or any other identifying information to join.

The privacy architecture is detailed in Chapter 05. Messages carry only what is necessary for routing. No phone numbers, no email addresses, no persistent identifiers. Multi-profile identities let users compartmentalize their digital lives, keeping work, personal, and sensitive communication entirely separate. Traffic obfuscation through cover traffic and timing randomization makes traffic analysis significantly harder.

Privacy advocates can use the AI routing system knowing that no raw message content is ever analyzed. The on-device inference described in Chapter 16 means that the AI helps route messages without reading them. Users who opt in to telemetry share only aggregate observations, never individual conversations.

3. Rural Communities

Tagline: "2.6 billion people lack reliable internet"

Rural communities around the world lack dependable internet access. GhostWire enables community-owned mesh networks using LoRa and WiFi. The AI routing system can be trained on datasets like the GuifiSants dataset, which contains 7,931 samples from 63 nodes over 31 days, to optimize routing for real-world rural topologies.

Community members can share news, alerts, and resources without relying on an internet service provider. The mesh is owned and operated by the community itself. Each device is a node, and the more people who join, the stronger the network becomes. The hardware ladder described in Chapter 15 shows how even a $20 ESP32 can participate as a sensor or leaf node, making the technology accessible to communities with limited budgets.

The AI routing model trained on the GuifiSants dataset learns optimal paths through specific terrain. This means that rural deployments benefit from routing intelligence that understands real-world conditions, not just theoretical models. The GNN model generalizes from the Barcelona mesh data to other community networks, adapting to local geography and node density.

4. Activists and Journalists

Tagline: "Coordinate without infiltration. Disposable identities. Mesh during protests."

Activists and journalists face internet shutdowns, surveillance, and censorship. During protests, governments shut down internet access. Journalists lose their ability to communicate with editors. Activists are infiltrated by informants. Standard communication tools become liabilities.

GhostWire provides mesh networking that works during internet shutdowns using WiFi Direct and Bluetooth Low Energy. Disposable identities allow users to create separate profiles for different operations and burn them when compromised. QR pairing enables secure key exchange with trusted contacts in person, with no central registry to subpoena.

Sybil resistance through proof-of-work node admission makes it computationally expensive for adversaries to flood the mesh with fake nodes. The TrustStore reputation system tracks node behavior over time and flags suspicious nodes automatically. These security features are described in Chapter 05.

5. Whistleblowers

Tagline: "Send documents without revealing identity. Tor integration supported."

Whistleblowers need to transmit sensitive documents without revealing their identity. Standard channels leave traces. Even encrypted services can be compelled to hand over metadata. GhostWire provides several features that address this need.

Tor integration is planned and will route GhostWire traffic through the Tor network for additional anonymity layers. In the meantime, stealth TCP provides obfuscated transport that can blend GhostWire traffic with normal HTTPS traffic, making it harder to detect. Ephemeral sessions ensure that session keys are destroyed after use, leaving no persistent state to subpoena.

No identifying information is required to create a GhostWire identity. No phone number, email, or real name is needed. The end-to-end encryption described in Chapter 05 ensures that documents are encrypted before they leave the device, and no intermediary can read them.

6. Developers and Researchers

Tagline: "AGPL-licensed. Built in Rust. AI models trained on real mesh data."

Developers and researchers can contribute to GhostWire because the project is licensed under the GNU Affero General Public License version 3 (AGPLv3). The codebase is written in Rust for memory safety. The architecture supports pluggable transports and an adapter registry that allows developers to build bridges to other protocols.

The technology stack includes libp2p networking, Kademlia DHT, Gossipsub pub/sub, QUIC and TCP transports, and a React web dashboard. The AI layer includes LightGBM anomaly detection exported to ONNX and a Graph Neural Network trained on real mesh data. Researchers can study the routing algorithms, test new transport modules, and contribute to the AI models.

The adapter registry described in Chapter 04 provides a structured way to add new transports and protocols. Developers can implement the relevant adapter interface and register it without modifying the core codebase. This modular design makes GhostWire a fertile ground for research into mesh networking, privacy-preserving communication, and AI-driven routing.

---

2. Persona-to-Capability Map

The diagram below maps each persona to the GhostWire capabilities they rely on. Cross-references point to the chapters that describe those capabilities in detail.

```
graph TD;
    Disaster["1. Disaster Responders"] -->|"WiFi/BLE Mesh, LoRa, Delay-Tolerant Delivery"| Cap
    Privacy["2. Privacy Advocates"] -->|"Metadata Minimization, Multi-Profile Identities"| Cap
    Rural["3. Rural Communities"] -->|"Community Mesh, AI Routing on GuifiSants Data"| Cap3["C
    Activist["4. Activists and Journalists"] -->|"QR Pairing, Sybil PoW, TrustStore"| Cap4["Cha
```

```
Whistle["5. Whistleblowers"] -->|"Tor Integration, Stealth TCP, Ephemeral Sessions"| Cap5["
Dev["6. Developers and Researchers"] -->|"AGPLv3, Rust Stack, Adapter Registry"| Cap6["Cha

Disaster -->|"Battery-Aware Scaling"| Cap7["Chapter 15: Hierarchy"]
Rural -->|"Hardware Ladder"| Cap7
Activist -->|"Privacy Tiers"| Cap7

style Disaster fill:#f9f,stroke:#333,stroke-width:2px
style Privacy fill:#bbf,stroke:#333,stroke-width:2px
style Rural fill:#bfb,stroke:#333,stroke-width:2px
style Activist fill:#fbb,stroke:#333,stroke-width:2px
style Whistle fill:#fbf,stroke:#333,stroke-width:2px
style Dev fill:#ff9,stroke:#333,stroke-width:2px
```

The table below provides a more detailed mapping of features to personas.

Feature	Disaster Responders	Privacy Advocates	Rural Communities	Activists and Journalists	Whistleblowers	Developers and Researchers
WiFi Direct / BLE Mesh	Yes	Yes	Yes	Yes	Yes	Yes
LoRa	Yes	No	Yes	Yes	No	Yes
Delay-Tolerant Delivery	Yes	No	Yes	Yes	Yes	Yes
Metadata Minimization	No	Yes	No	Yes	Yes	Yes
Multi-Profile Identities	No	Yes	No	Yes	Yes	Yes
QR Pairing	No	No	No	Yes	Yes	Yes
Sybil Proof-of-Work	No	No	No	Yes	No	Yes
TrustStore Reputation	No	No	No	Yes	No	Yes
Tor Integration (planned)	No	No	No	No	Yes	Yes
Stealth TCP	No	Yes	No	Yes	Yes	Yes
Ephemeral Sessions	No	No	No	No	Yes	Yes
Pluggable Transports	Yes	Yes	Yes	Yes	Yes	Yes
Adapter Registry	No	No	No	No	No	Yes

3. Four Illustrative Scenarios

The following scenarios are illustrative examples drawn from the project's documented use cases. They show how GhostWire might work in practice for different communities. These are not descriptions of completed deployments but rather plausible scenarios based on the project's stated goals and capabilities. Chapter 13 frames these same scenarios in detail and is cross-referenced below.

Scenario 1: Community Blackout with Protesters

During a citywide internet blackout, a community forms a mesh using phones and laptops. Bluetooth and WiFi Direct connect nearby devices, and a few laptops act as relays. Messages flow through the mesh without any internet connection. Activists coordinate protests, share safety alerts, and maintain communication with external journalists. This scenario is explored in more detail in the Case Studies chapter, which frames it as a community network during a communication blackout.

Scenario 2: Hurricane Disaster Response

After a hurricane destroys infrastructure, responders deploy LoRa nodes at shelters and staging areas. Battery-powered devices communicate over long distances. Store-and-forward delivery ensures that messages reach their destination even when nodes are temporarily out of range. The AI routing system helps messages find the best available path across the mesh. The Case Studies chapter describes this as a disaster response in a blackout, with responders operating 200 kilometers from infrastructure.

Scenario 3: Rural Village Connectivity

A village with no internet access connects homes, a school, and a clinic using LoRa and WiFi. A Raspberry Pi gateway bridges the local mesh to the wider network when a satellite internet connection is available. The AI routing system, trained on real-world rural data including the GuifiSants dataset, optimizes message paths. Community members share news, alerts, and educational resources. The Case Studies chapter frames this as rural village connectivity, showing how community-owned mesh transforms isolated areas.

Scenario 4: Festival

At a large outdoor festival, attendees use GhostWire to share updates and stay connected when cell networks become overloaded. Phones form a Bluetooth and WiFi mesh, and information spreads quickly through the crowd. The tiered architecture from Chapter 15 ensures that the mesh remains stable even as hundreds of devices join simultaneously. The Case Studies chapter describes this as a community event mesh, demonstrating how GhostWire handles congestion during large gatherings.

---

4. Who Is Behind GhostWire

GhostWire is built by Team GhostWire, a group of seven members affiliated with the Open University of Kenya. The project lead is Michael Irungu, who oversees system architecture, the Rust core, AI

and machine learning integration, and cryptographic design. The team submitted GhostWire to the Global Challenge Design for Future 2026 competition, a program that challenges teams to build technology solutions that address real-world problems with AI.

The project aligns with three United Nations Sustainable Development Goals. SDG 9 focuses on industry, innovation, and infrastructure, recognizing that connectivity is a foundation for economic development. SDG 11 focuses on sustainable cities and communities, acknowledging that urban and rural populations alike need resilient communication systems. SDG 17 focuses on partnerships for the goals, reflecting the collaborative nature of the project across institutions and disciplines.

The project's website carries a line that reflects its philosophy: "The internet was built to be ungovernable." This idea drives the design of a network that no single entity controls. The team believes that communication should be a right, not a privilege, and that the infrastructure for communication should be owned by the people who use it. The Ubuntu spirit guides this work: restoration, not innovation. GhostWire does not invent something new. It remembers something old, that communication belongs to those who communicate.

The team members are Michael Irungu, Lilian, Mark, Felix, Kibwana, Austin, and Frank. All are affiliated with the Open University of Kenya. The project is open source and welcomes contributions from developers, researchers, and community members around the world.

---

5. Which Persona Should Read Which Chapter

Different readers will find different chapters most useful. The table below helps each persona navigate the GhostWire Bible to the sections most relevant to their needs.

Persona	Start Here	Then Read	Also Relevant
Disaster Responders	Chapter 04 (Transports)	Chapter 15 (Hierarchy)	Chapter 05 (Security), Chapter 13 (Case Studies)
Privacy Advocates	Chapter 05 (Security)	Chapter 16 (AI Routing)	Chapter 04 (Transports), Chapter 13 (Case Studies)
Rural Communities	Chapter 16 (AI Routing)	Chapter 15 (Hierarchy)	Chapter 04 (Transports), Chapter 13 (Case Studies)
Activists and Journalists	Chapter 05 (Security)	Chapter 04 (Transports)	Chapter 15 (Hierarchy), Chapter 13 (Case Studies)
Whistleblowers	Chapter 05 (Security)	Chapter 04 (Transports)	Chapter 13 (Case Studies)
Developers and Researchers	Chapter 04 (Transports)	Chapter 06 (Adapters)	Chapter 12 (Developer Guide), Chapter 16 (AI Routing)
General Readers	Chapter 01 (Project Overview)	Chapter 07 (FAQ)	Chapter 13 (Case Studies), Chapter 14 (Glossary)

End of Chapter